Rdzeń

Formalna definicja języka programowania

Wersja 0.6 — szkic normatywny

29 października 2026

Dokument definiuje składnię, statyczną semantykę, semantykę weryfikacyjną (warunki weryfikacji), dynamiczną semantykę oraz format diagnostyki języka **Rdzeń** — małego, statycznie typowanego języka ogólnego przeznaczenia, zaprojektowanego tak, aby jego pełna specyfikacja mieściła się w kontekście modelu językowego, a kompilator maksymalizował automatycznie sprawdzane gwarancje poprawności i informację zwrotną o błędach.

Definicja ma postać jądra kalkulusowego λ_{Rz} oraz elaboracji pełnej składni do jądra.

Spis treści

1 Wprowadzenie ... 3
1.1 Status i zakres ... 3
1.2 Zasady projektowe ... 3
1.3 Konwencje notacyjne ... 4
2 Składnia konkretna ... 4
2.1 Leksyka ... 4
2.2 Reguła układu ... 4
2.3 Gramatyka ... 5
2.4 Priorytety i łączność ... 6
2.5 Forma kanoniczna ... 6
2.6 Odstępstwa od języków wzorcowych ... 7
3 Jądro λ_{Rz} ... 7
3.1 Typy ... 7
3.2 Wyrażenia ... 8
3.3 Operacje pierwotne ... 9
3.4 Deklaracje globalne ... 9
4 Elaboracja ... 9
4.1 Bloki, wiązania i sterowanie ... 9
4.2 Operator ? ... 10
4.3 Pętle ... 10
4.4 Uściślenia jako zobowiązania ... 10
4.5 Kontrakty, przykłady i właściwości ... 11
4.6 Domknięcia ... 11
4.7 Aktorzy ... 12
5 Statyczna semantyka ... 12
5.1 Konteksty i osądy ... 12
5.2 Reguły typowania wyrażeń ... 12
5.3 Deklaracje i totalność ... 14
5.4 Prawo wyłączności ... 15
5.5 Polimorfizm, ograniczenia i inferencja ... 15
5.6 Wyczerpywalność dopasowań ... 15
5.7 Kapabilności i brak władzy otoczenia ... 15
6 Semantyka weryfikacyjna ... 16
6.1 Fragment logiki F ... 16
6.2 Dobra określoność formuł ... 17
6.3 Hipotezy typów ... 17
6.4 Warunki weryfikacji ... 17
6.5 Rodzaje zobowiązań ... 20
6.6 Rozstrzyganie zobowiązań ... 20
6.7 Zaufana baza ... 21
7 Dynamiczna semantyka ... 22
7.1 Konfiguracje i zdarzenia ... 22
7.2 Konteksty obliczeń ... 22
7.3 Reguły redukcji ... 22
7.4 Aktorzy ... 24
7.5 Zachowanie obserwowalne ... 24
8 Metateoria ... 24
8.1 Bezpieczeństwo pamięci i brak aliasingu ... 24
8.2 Poprawność weryfikacji ... 25

8.3 Efekty, terminacja, wykonywalność ... 26
8.4 Status dowodów ... 27
8.5 Mechanizacja (Lean 4) ... 27
9 Diagnostyka ... 28
9.1 Rekord diagnostyki ... 28
9.2 Klasy kodów ... 28
9.3 Brak kaskady ... 29
9.4 Kontrprzykłady ... 29
9.5 Raport dziury ... 29
9.6 Raport weryfikacji ... 29
9.7 Przykłady diagnostyk ... 29
A Przykład kompletny: wyszukiwanie binarne ... 31
A.1 Program ... 31
A.2 Fragment elaboracji ... 32
A.3 Zobowiązania ... 32
B Preludium ... 33

1 Wprowadzenie

1.1 Status i zakres

Dokument jest **normatywny** w rozdziałach 2–8 i **informacyjny** w dodatkach. Definicja obejmuje wersję 0.1 języka. Elementy oznaczone jako *poza wersją 0.1* są zarezerwowane składniowo, ale nie mają semantyki.

Definicja ma budowę warstwową, wzorowaną na definicji Standard ML:

1. **Składnia konkretna** (rozdz. 2): leksyka, reguła układu, gramatyka, forma kanoniczna.
2. **Jądro** λ_{Rz} (rozdz. 3): mały kalkulus, do którego elaborowana jest pełna składnia.
3. **Elaboracja** (rozdz. 4): tłumaczenie składni konkretnej na jądro, w tym wstawianie zobowiązań dowodowych.
4. **Statyczna semantyka** (rozdz. 5): typy kształtu, efekty, wyłączność dostępu, wyczerpywalność, totalność.
5. **Semantyka weryfikacyjna** (rozdz. 6): fragment logiki, warunki weryfikacji, rozstrzyganie zobowiązań, weryfikacja stopniowana.
6. **Dynamiczna semantyka** (rozdz. 7): semantyka operacyjna small-step z etykietowanymi zdarzeniami, aktorzy.
7. **Metateoria** (rozdz. 8): twierdzenia o poprawności z ich statusem dowodowym.
8. **Diagnostyka** (rozdz. 9): normatywny format komunikatów kompilatora.

1.2 Zasady projektowe

Z1 **Małość.** Specyfikacja z biblioteką standardową w postaci sygnatur mieści się w ok. 25 tys. tokenów.

Z2 **Jeden sposób zapisu.** Program jest poprawny składniowo tylko w formie kanonicznej (§2.5).

Z3 **Brak fałszywych przyjaciół.** Konstrukcja zapisana tak jak w Kotlinie lub Ruście ma tę samą semantykę. Gdzie semantyka się różni, zapis jest inny. Odstępstwa wymienia §2.6.

Z4 **Jawność.** Brak niejawnych konwersji, przeciążania, dziedziczenia, makr, refleksji, wyjątków, wartości `null` i globalnego stanu mutowalnego.

Z5 **Lokacje nie są wartościami.** Mutowalne zmienne nie mogą być przechowywane, zwracane ani przechwytywane. Jedynym sposobem współdzielenia lokacji jest parametr `inout`, objęty prawem wyłączności. Z tego wynika brak aliasingu (§8).

Z6 **Każda formuła jest wykonywalna.** Fragment logiki specyfikacji zawiera wyłącznie kwantyfikatory ograniczone, więc każdy kontrakt można sprawdzić w czasie wykonania (tw. 8.8). To umożliwia weryfikację stopniowaną. Wyjątkiem są hipotezy typów funkcyjnych (kwantyfikacja po całej dziedzinie); te sprawdza się leniwie — monitorem przy każdym wywołaniu (§6.6).

Z7 **Determinizm diagnostyki.** Ten sam program daje tę samą listę diagnostyk w tej samej kolejności, niezależnie od maszyny i czasu.

1.3 Konwencje notacyjne

- $\overline{x}$ oznacza skończony ciąg $x_1, \ldots, x_n$; $|\overline{x}|$ to jego długość.
- Γ — kontekst typowania, Δ — kontekst zmiennych typowych i efektowych, Λ — typowanie lokacji (pamięci), Σ — sygnatury globalne.
- $e[v/x]$ — podstawienie bez przechwytywania zmiennych.
- φ, ψ — formuły logiki specyfikacji; $\vDash \varphi$ — prawdziwość w modelu standardowym.
- $\to$ — krok redukcji, $\overset{*}{\to}$ — domknięcie zwrotno-przechodnie, $\rightsquigarrow$ — elaboracja.
- Słowa kluczowe składni konkretnej pisane są czcionką maszynową (`while`), a w jądrze pismem prostym (while).

2 Składnia konkretna

2.1 Leksyka

Kodowanie. Plik źródłowy jest poprawnym tekstem UTF-8 z rozszerzeniem `.rz`. Znaki spoza ASCII mogą występować wyłącznie w literałach napisowych i komentarzach.

Identyfikatory. Kategoria leksykalna rozróżnia rolę nazwy, co usuwa niejednoznaczność wzorców (zmienna czy konstruktor):

```
ident   = [a-z_] [a-zA-Z0-9_]*        (* wartości, funkcje, pola *)
Tident  = [A-Z]  [a-zA-Z0-9]*         (* typy, konstruktory, rodzaje kapabilności *)
hole    = "?" [a-z_] [a-zA-Z0-9_]*    (* dziury *)
```

Słowa kluczowe (zarezerwowane, nie mogą być identyfikatorami):

```
actor  as  assert  break  const  continue  data  decreases  else  ensures
examples  exists  extern  false  fn  for  forall  if  import  in  inout
invariant  lemma  let  match
old  partial  pub  receives  recv  requires  result  return  self  send  spawn  spec
true  type  var  where  while
```

Literały.

- całkowite: `0`, `42`, `1_000_000`, `0xFF`, `0b1010` (separator `_` tylko między cyframi);
- bajtowe: `b'a'`, `b'\n'` — wartość typu `Byte` (literał musi oznaczać dokładnie jeden bajt UTF-8, inaczej `E0013`);
- zmiennoprzecinkowe: `1.5`, `2.0e-3` (zawsze z kropką i cyfrą po obu stronach);
- napisowe: `"..."` z sekwencjami `\n \t \r \0 \\ \" \' \u{XXXX}` (nieznana sekwencja: `E0014`); literał oznacza tablicę bajtów swojego kodowania UTF-8, np. `"ść"` ma długość 4.

Komentarze. `//` do końca wiersza; `///` komentarz dokumentacyjny, przypisany do następnej deklaracji. Komentarzy blokowych nie ma (Z2).

2.2 Reguła układu

Średniki nie są pisane. Lekser wstawia token `;` na końcu wiersza wtedy i tylko wtedy, gdy ostatni token w tym wierszu należy do zbioru

```
ident  Tident  hole  literał  )  ]  }  return  break  continue  ?  true  false
```

Reguła jest identyczna z regułą wstawiania średników w języku Go. Wynika z niej, że operator dwuargumentowy kontynuujący wyrażenie musi stać na końcu wiersza, a nie na początku następnego.

2.3 Gramatyka

Gramatyka jest LL(1) po wstawieniu średników. Notacja: `{ x }` — zero lub więcej, `[ x ]` — opcjonalne, `|` — alternatywa.

```
program     = { item } ;
item        = [ "pub" ] ( fn_decl | data_decl | type_decl | spec_decl
                        | const_decl | extern_decl | actor_decl ) | import ;
import      = "import" path [ "as" ident ] ";" ;
path        = ident { "." ident } ;

(* ---- typy ---- *)
type        = base_type [ "where" formula ] ;
base_type   = Tident [ "<" type { "," type } ">" ]
            | "fn" "(" [ fparam { "," fparam } ] ")" "->" type [ "!" effects ] ;
fparam      = [ ident ":" ] type ;       (* nazwa: do uściśleń zależnych *)
effects     = "{" [ effect { "," effect } ] "}" ;
effect      = Tident | ident ;           (* etykieta lub zmienna efektowa *)
tparams     = "<" tparam { "," tparam } ">" ;
tparam      = Tident [ ":" ( "Eq" | "Ord" ) ] | ident ;   (* ident = zm. efektowa *)

(* ---- deklaracje ---- *)
data_decl   = "data" Tident [ tparams ] "{" ctor { "," ctor } [ "," ] "}"
              [ "where" formula ] ;
ctor        = Tident [ "(" field { "," field } ")" ] ;
field       = ident ":" type ;
type_decl   = "type" Tident [ tparams ] "=" type ";" ;
const_decl  = "const" ident ":" type "=" expr ";" ;   (* wyrażenie stałe *)
spec_decl   = "spec" "fn" ident [ tparams ] "(" [ params ] ")" "->" type [ ";" ]
              [ "decreases" term [ ";" ] ] "=" formula ";" ;
extern_decl = "extern" "fn" ident "(" [ params ] ")" [ "->" type ]
              "!" effects [ ";" ] { contract } ;
fn_decl     = [ "partial" | "lemma" ] "fn" ident [ tparams ] "(" [ params ] ")"
              [ "->" type ] [ "!" effects ] [ ";" ]
              { contract } [ examples [ ";" ] ] block ;
params      = param { "," param } ;
param       = [ "inout" ] ident ":" type ;
lparam      = ident [ ":" type ] ;       (* typ pominięty: z typu oczekiwanego *)
contract    = ( "requires" formula | "ensures" formula
              | "decreases" term ) ";" ;
examples    = "examples" "{" { ( expr | "forall" binders ":" expr ) ";" } "}" ;
binders     = ident ":" type { "," ident ":" type } ;
actor_decl  = "actor" ident "(" [ params ] ")" "receives" type block ;

(* ---- instrukcje i wyrażenia ---- *)
block       = "{" { stmt ";" } [ expr ] "}" ;
stmt        = "let" ident [ ":" type ] "=" expr
            | "var" ident [ ":" type ] "=" expr
            | lvalue "=" expr
            | "while" expr [ ";" ] { loop_ann } block
            | "for" ident "in" expr [ ";" ] { loop_ann } block
            | "assert" formula
            | "return" [ expr ] | "break" | "continue"
            | expr ;
loop_ann    = ( "invariant" formula | "decreases" term ) ";" ;
lvalue      = ident { "." ident } [ "[" expr "]" ] ;
```

```
expr        = or_expr ;
or_expr     = and_expr { "||" and_expr } ;
and_expr    = cmp_expr { "&&" cmp_expr } ;
cmp_expr    = add_expr [ ( "==" | "!=" | "<" | "<=" | ">" | ">=" ) add_expr ] ;
add_expr    = mul_expr { ( "+" | "-" ) mul_expr } ;
mul_expr    = unary { ( "*" | "/" | "%" ) unary } ;
unary       = [ "-" | "!" ] postfix ;
postfix     = primary { "(" [ args ] ")" | "[" expr "]" | "." ident | "?" } ;
args        = arg { "," arg } ;
arg         = expr | "&" ident ;                    (* & — argument inout *)
primary     = literal | ident | Tident [ "(" [ args ] ")" ] | hole
            | "(" expr ")" | "[" [ expr { "," expr } ] "]"
            | block | if_expr | match_expr
            | "fn" "(" [ lparam { "," lparam } ] ")" [ "->" type ] "=>" expr
            | "spawn" ident "(" [ args ] ")"
            | "send" "(" expr "," expr ")" | "recv" "(" "&" ident ")" ;
if_expr     = "if" expr block [ "else" ( if_expr | block ) ] ;
match_expr  = "match" expr "{" arm { "," arm } [ "," ] "}" ;
arm         = pattern [ "if" expr ] "=>" expr ;      (* instrukcje: w bloku { … } *)
pattern     = "_" | ident | literal | Tident [ "(" pattern { "," pattern } ")" ] ;

(* ---- formuły i termy specyfikacji (podzbiór wyrażeń) ---- *)
formula     = expr                                   (* ograniczenia: §6.1 *)
            | ( "forall" | "exists" ) ident "in" expr ".." expr ":" formula
            | "old" "(" ident ")" ;
term        = expr ;
```

Ograniczenia kontekstowe na `formula` i `term` definiuje §6.1. Kwantyfikatory `forall`, `exists` oraz `old(…)`, `result` i `self` są dopuszczalne wyłącznie w formułach.

Opcjonalne średniki w nagłówkach funkcji i pętli pozwalają umieścić kontrakty i adnotacje w osobnych wierszach (reguła układu wstawia średnik po ostatnim tokenie nagłówka). Gałąź `match` zawiera wyrażenie; instrukcje (przypisanie, `return`) zapisuje się w bloku: `Less => { lo = mid + 1 }`.

2.4 Priorytety i łączność

Od najsłabszego: `||`, `&&`, porównania (niełączne — `a < b < c` jest błędem składni), `+ -`, `* / %`, jednoargumentowe `- !`, przyrostkowe `() [] . ?`. Operatory dwuargumentowe łączą lewostronnie.

2.5 Forma kanoniczna

Definicja 2.1 (forma kanoniczna)**.** Niech $\mathrm{fmt} : \mathrm{AST} \to \mathrm{Tekst}$ będzie deterministyczną funkcją formatowania (wcięcie 4 spacje, jeden znak spacji wokół operatorów dwuargumentowych, maksymalna szerokość wiersza 100 znaków z deterministycznym łamaniem, kolejność importów leksykograficzna). Program P jest **kanoniczny**, gdy $\mathrm{fmt}(\mathrm{parse}(P)) = P$.

Drzewo AST obejmuje tu komentarze (przypisane jednostkom: deklaracji, klauzuli, instrukcji, gałęzi `match`, konstruktorowi), zapis literałów całkowitych (`0x…`, `1_000`, `b'a'`) oraz informację o pustych wierszach między instrukcjami. Funkcja fmt jest wyznaczona regułami:

1. Klauzule `requires`, `ensures`, `decreases` (w tej kolejności), blok `examples` oraz adnotacje pętli `invariant`, `decreases` stoją w osobnych wierszach z wcięciem o jeden poziom; klamra otwierająca ciało stoi w osobnym wierszu, jeśli występuje co najmniej jedna klauzula, w przeciwnym razie kończy wiersz nagłówka.
2. `if` i `match` w pozycji instrukcji są zawsze wielowierszowe (`} else if … {` i `} else {` w jednym wierszu; gałęzie `match` zakończone przecinkiem). W pozycji wyrażenia są jednowierszowe, jeśli mieszczą się w wierszu, nie zawierają komentarzy, a gałęzie są wyrażeniami albo blokami z jedną prostą instrukcją (`Some(v) => { n = v }`).

3. Nawiasy są wypisywane tam, gdzie wymaga ich odtworzenie drzewa (§2.4), oraz dla czytelności: wokół `&&` będącego argumentem `||` i wokół `if`/`match` będących argumentem operatora. Grupowanie zapisane przez programistę, które zmienia drzewo (np. `a - (b - c)`), jest zachowane.
4. Wiersz dłuższy niż 100 znaków jest łamany kolejno: po operatorach łańcucha `&&`/`||` (operator na końcu wiersza, zgodnie z regułą układu), po argumentach wywołania (jeden na wiersz, z przecinkiem końcowym), po dwukropku kwantyfikatora lub strzałce domknięcia; nagłówek funkcji — po parametrach (jeden na wiersz).
5. Deklaracje są oddzielone jednym pustym wierszem, z wyjątkiem kolejnych deklaracji jednowierszowych bez pustego wiersza w źródle (grupa). Wewnątrz bloku zachowywany jest najwyżej jeden pusty wiersz między instrukcjami; nigdy bezpośrednio po `{` ani przed `}`.
6. Komentarz w osobnym wierszu poprzedza swoją jednostkę i ma jej wcięcie; komentarz końcowy zostaje na końcu wiersza swojej jednostki, oddzielony dwiema spacjami.

Kompilator odrzuca program niekanoniczny błędem `E0100`, załączając jako poprawkę wynik $\mathrm{fmt}(\mathrm{parse}(P))$ (różnicę wierszy); `rzc fmt -w` zapisuje postać kanoniczną.

2.6 Odstępstwa od języków wzorcowych

Zgodnie z Z3 poniższe konstrukcje mają semantykę **inną** niż w Kotlinie lub Ruście, więc ich zapis albo jest identyczny z językiem o tej samej semantyce, albo jest nowy:

Konstrukcja	Semantyka w Rdzeniu
`a + b` na `Int`	Przepełnienie powoduje pułapkę `overflow` (jak Rust w trybie debug i Swift). Kotlin zawija — w Rdzeniu nie ma operatora zawijającego, jest funkcja `wrapping_add`.
`a / b`, `a % b`	Obcinanie w stronę zera (jak Kotlin, Rust, C, Go; inaczej niż Python).
`==`	Równość strukturalna. Niezdefiniowana dla typów funkcyjnych (błąd typu).
`let` / `var`	`let` jak w Ruście (niemutowalne wiązanie). `var` jak w Kotlinie, ale lokacja nie może uciec z zasięgu (Z5).
`&x`	**Nie** jest referencją ani wskaźnikiem. Oznacza wyłącznie przekazanie zmiennej jako argumentu `inout` (semantyka Swiftowego `inout`).
`?`	Jak w Ruście: wczesny powrót z `None` lub `Err(e)`, bez konwersji typu błędu.

3 Jądro λ_{Rz}

Jądro jest językiem pośrednim, do którego elaborowany jest każdy program (rozdz. 4). Wszystkie pozostałe rozdziały normatywne definiują semantykę na poziomie jądra.

3.1 Typy

```
K   ::= Io | Fs | Net | Clock | Rand | Ffi | Spawn      rodzaje kapabilności
ℓε  ::= K | Div                                         etykiety efektów
ε   ::= ∅ | {ℓε} | ρ | ε ∪ ε                            efekty (ρ — zmienna)

B   ::= Int | Bool | Byte | F64 | Unit | Str            typy bazowe
τ   ::= B                                               typy kształtu
     |  T⟨τ⟩                                            typ danych
     |  Array⟨τ⟩
     |  α                                               zmienna typowa
     |  (x̄ : T̄) →ε T                                   funkcja (bez inout); x̄ wiążą w T
     |  Cap⟨K⟩ | Chan⟨τ⟩ | Mailbox⟨τ⟩
T   ::= {ν : τ | φ}                                     typ uściślony, φ ∈ F
```

Typ uściślony $\{\nu : \tau \mid \top\}$ utożsamiamy z τ. Składnia konkretna `τ where φ` oznacza $\{\nu : \tau \mid \varphi[\nu/\text{ self}]\}$. Funkcja $\text{erase}(\{\nu : \tau \mid \varphi\}) = \tau$ usuwa uściślenia; statyczna semantyka (rozdz. 5) operuje na typach kształtu, a uściślenia są obsługiwane wyłącznie przez semantykę weryfikacyjną (rozdz. 6).

W typie funkcyjnym $(\overline{x} : \overline{T}) \xrightarrow{\varepsilon} \{\nu : \sigma \mid \psi\}$ uściślenie wyniku ψ może odwoływać się do parametrów $\overline{x}$ (uściślenie zależne), np. `fn(x: Int) -> Int where self > x`. Uściślenia parametrów są warunkiem wstępnym wartości funkcyjnej, uściślenie wyniku — jej gwarancją; funkcja nazwana z `requires` nie jest wartością (warunek należy zapisać w typie parametru).

Typy `Nat`, `Byte`, `Str` i `Ordering` są zdefiniowane w preludium (dodatek B): `type Nat = Int where self >= 0`, `type Byte = Int where 0 <= self && self < 256`, `type Str = Array<Byte>`. `Byte` i `Str` w produkcji B są więc skrótami: napis jest tablicą bajtów UTF-8 bez osobnego typu kształtu, a wszystkie operacje i zobowiązania tablic stosują się do napisów bez zmian.

3.2 Wyrażenia

Zmienne dzielą się na trzy rozłączne kategorie syntaktyczne, co jest podstawą twierdzenia o braku aliasingu:

- x, y, z — zmienne **niemutowalne** (wiązane przez `let`, parametry wartościowe, wzorce);
- m — zmienne **mutowalne** (wiązane przez `var` i parametry `inout`);
- ℓ — **lokacje**, występujące wyłącznie w konfiguracjach czasu wykonania, nigdy w programie źródłowym.

```
v  ::= n | b | () | c | s                                  literały
    |  C⟨τ⟩(v̄)                                            konstruktor
    |  [v̄]                                                tablica
    |  fn(x̄ : τ) ⇒ e     gdzie fv(e) ⊆ {x̄}               domknięcie zamknięte
    |  cap(K) | chan(a) | mbox(a)

μ  ::= m | ℓ                                              cele mutacji

e  ::= v | x | f
    |  !μ                                                  odczyt
    |  μ := e  |  μ[e] := e                               zapis, zapis elementu
    |  let x = e in e  |  var m : τ = e in e
    |  op(ē)                                               operacja pierwotna
    |  C⟨τ⟩(ē)  |  [ē]
    |  if e then e else e
    |  match e { p̄ when ḡ ⇒ ē }
    |  while e inv φ⁻dec t do e
    |  break  |  continue  |  return e
    |  f⟨τ̄;ε⟩(ā)                                          wywołanie globalne
    |  e(ē)                                                wywołanie domknięcia
    |  fn(x̄ : τ) ⇒ e
    |  assert[κ] φ                                         zobowiązanie
    |  check[κ] φ                                          sprawdzenie dynamiczne
    |  snap[κ] t₀ := t in e                                migawka wariantu
    |  trap(k)  |  hole[h]                                 pułapka, dziura
    |  spawn f(ā)  |  send(e, e)  |  recv(μ)

a  ::= e | &μ                                             argument (&μ: inout)
p  ::= _ | x | n | b | C(p̄)                               wzorce
k  ::= overflow | div0 | bounds | contract(κ)              rodzaje pułapek
    |  unreachable | stack

e  ::= … | ret{e} | brk{e} | cnt{e} | iter(e, e)          ramki (tylko w czasie
                                                          wykonania)
```

Identyfikatory zobowiązań κ są unikalne w programie i niosą rodzaj zobowiązania (§6.5) oraz pozycję w źródle.

3.3 Operacje pierwotne

```
arytmetyka   add sub mul div rem : (Int, Int) → Int     pułapki: overflow, div0
             neg : (Int) → Int                          pułapka: overflow
porównania   eq ne lt le gt ge                          eq, ne: typy Eq
logika       not                                        && i || — §4.1
tablice      len  : (Array⟨τ⟩) → Int
             get  : (Array⟨τ⟩, Int) → τ                 pułapka: bounds
             push : (inout Array⟨τ⟩, τ) → Unit
             pop  : (inout Array⟨τ⟩) → Option⟨τ⟩
napisy       str_concat, str_eq, parse_int, …           dodatek B
kapabilności print : (Cap⟨Io⟩, Str) →{Io} Unit
             read_line : (Cap⟨Io⟩) →{Io} Option⟨Str⟩
             now : (Cap⟨Clock⟩) →{Clock} Int
             random : (Cap⟨Rand⟩, Int, Int) →{Rand} Int
```

Operacje `push` i `pop` przyjmują tablicę jako argument `inout`; w jądrze zapisuje się je $\mathrm{push}(\&\mu, e)$. Pełny wykaz operacji z kontraktami zawiera dodatek B.

3.4 Deklaracje globalne

Program jądra to sygnatura Σ i zbiór definicji. Funkcja globalna ma postać

$$f\langle\overline{\alpha},\overline{\rho}\rangle(\overline{x}:\overline{\tau},\mathrm{inout}\ \overline{m}:\overline{\sigma})\to\tau!\varepsilon\quad\mathrm{requires}\ P\quad\mathrm{ensures}\ Q\quad\mathrm{decreases}\ t\quad=e$$

gdzie: parametry wartościowe $\overline{x}$ poprzedzają parametry `inout` $\overline{m}$ (elaboracja porządkuje parametry i dopasowuje argumenty; kolejność źródłowa jest zachowana w diagnostyce); P może mówić o $\overline{x}, \overline{m}$; Q może mówić o $\overline{x}, \overline{m}$ (stan końcowy), $\mathrm{old}(m_i)$ (stan początkowy) oraz result; t jest termem fragmentu całkowitoliczbowego, obecnym wtedy i tylko wtedy, gdy f należy do nietrywialnej silnie spójnej składowej grafu wywołań i $\mathrm{Div} \notin \varepsilon$.

Funkcja `extern` ma tę samą sygnaturę bez ciała; jej kontrakt jest **założeniem zaufanym** (§6.7) i jej efekt musi zawierać Ffi.

Typ danych ma postać data $T\langle\overline{\alpha}\rangle = C_1(\overline{\tau_1}) \mid \ldots \mid C_n(\overline{\tau_n})$ where φ_T; niezmiennik φ_T (domyślnie $\top$) obowiązuje każdą wartość typu T (§4.4). Typy danych mogą być rekurencyjne; ponieważ wartości są drzewami (brak lokacji w wartościach), nie mogą tworzyć cykli.

4 Elaboracja

Elaboracja jest funkcją częściową $\mathcal{E}$ z poprawnie sformułowanych (kanonicznych, rozwiązanych nazwowo) programów konkretnych w programy jądra. Jest zdefiniowana przez przypadki poniżej; przypadki nie wymienione tłumaczą się homomorficznie. Elaboracja jest kierowana typami: korzysta z typów kształtu wyznaczonych przez statyczną semantykę (rozdz. 5), którą wykonuje się na składni konkretnej z tymi samymi regułami.

Elaboracja wstawia trzy rodzaje węzłów: $\mathrm{assert}[\kappa]\varphi$ (zobowiązania), snap (migawki wariantów) oraz ramki sterowania. Nie wstawia węzłów check — te powstają dopiero w fazie rozstrzygania (§6.6).

4.1 Bloki, wiązania i sterowanie

Składnia konkretna	Jądro
`{ s; rest }` gdzie `s = let x = e`	$\mathrm{let}\ x = \mathcal{E}(e)\ \mathrm{in}\ \mathcal{E}(\{\mathrm{rest}\})$
`{ s; rest }` gdzie `s = var m = e`	$\mathrm{var}\ m : \tau = \mathcal{E}(e)\ \mathrm{in}\ \mathcal{E}(\{\mathrm{rest}\})$
`{ s; rest }` gdzie `s` to wyrażenie	$\mathrm{let}\ _ = \mathcal{E}(s)\ \mathrm{in}\ \mathcal{E}(\{\mathrm{rest}\})$
`{ }` (pusty blok)	$()$
`{ e }`	$\mathcal{E}(e)$

`m = e` (`m` mutowalna)	$m := \mathcal{E}(e)$
`m[i] = e`	$m[\mathcal{E}(i)] := \mathcal{E}(e)$
`m.f = e` (typ jednokonstruktorowy `C(f₁…fₙ)`)	$m := \text{match } !m\{C(\overline{y}) \Rightarrow C(y_1, ..., \mathcal{E}(e), ..., y_n)\}$
`x` (`x` mutowalna)	$!x$
`if c { a }` (bez `else`)	$\text{if } \mathcal{E}(c) \text{ then } \mathcal{E}(a) \text{ else } ()$
`a && b`	$\text{if } \mathcal{E}(a) \text{ then } \mathcal{E}(b) \text{ else false}$
`a || b`	$\text{if } \mathcal{E}(a) \text{ then true else } \mathcal{E}(b)$
`e.f` (pole typu jednokonstruktorowego)	$\text{match } \mathcal{E}(e)\{C(\overline{y}) \Rightarrow y_i\}$
`a[i]`	$\text{get}(\mathcal{E}(a), \mathcal{E}(i))$
`a.len`	$\text{len}(\mathcal{E}(a))$
`return` (bez wyrażenia)	$\text{return } ()$

Kolejność obliczania argumentów, operandów i pól jest **od lewej do prawej**. Zmienna mutowalna użyta jako argument wartościowy jest odczytywana (kopia wartości) w momencie obliczania tego argumentu.

4.2 Operator `?`

Dla `e?` z `e : Option⟨τ⟩` w funkcji o typie wyniku `Option⟨σ⟩`:

$$\mathcal{E}(e?) = \text{match } \mathcal{E}(e)\{\text{Some}(x) \Rightarrow x, \text{None} \Rightarrow \text{return None}\}$$

Dla `e : Result⟨τ, ε⟩` w funkcji o wyniku `Result⟨σ, ε⟩` (ten sam typ błędu — brak niejawnej konwersji, Z4):

$$\mathcal{E}(e?) = \text{match } \mathcal{E}(e)\{\text{Ok}(x) \Rightarrow x, \text{Err}(y) \Rightarrow \text{return Err}(y)\}$$

Użycie `?` w innym kontekście jest błędem `E2140`.

4.3 Pętle

Pętla `while` przenosi adnotacje bez zmian:

$$\mathcal{E}(\text{while } c \text{ invariant } \overline{\varphi} \text{ decreases } t\{b\}) = \text{while } \mathcal{E}(c) \text{ inv } \overline{\varphi} \text{ dec } t \text{ do } \mathcal{E}(b)$$

Brak `decreases` oznacza $t = \bot$ (pętla bez wariantu), co wymaga Div w efekcie otaczającej funkcji (§5.3). Warunek pętli musi być czysty (efekt $\varnothing$, brak zapisu).

Pętla `for x in a` po tablicy elaboruje się z **automatycznymi** adnotacjami, do których dołącza się adnotacje użytkownika $\overline{\varphi}$ (w których `x` jest niedostępne, a dostępny jest licznik pod nazwą `it`):

$$\begin{aligned}\mathcal{E}(\text{for } x \text{ in } a \text{ invariant } \overline{\varphi}\{b\}) = \text{let } a_0 = \mathcal{E}(a) \text{ in var it} : \text{Int} = 0 \text{ in}\\ \text{while lt}(!\text{it}, \text{len}(a_0))\\ \text{inv } (0 \le \text{it} \le \text{len}(a_0)), \overline{\varphi} \quad \text{dec len}(a_0) - \text{it}\\ \text{do } (\text{let } x = \text{get}(a_0, !\text{it}) \text{ in } \mathcal{E}(b); \text{it} := \text{add}(!\text{it}, 1))\end{aligned}$$

Tablica jest migawkowana (a_0), więc modyfikacja źródłowej zmiennej w ciele pętli nie wpływa na iterację. `continue` w ciele `for` elaborowane jest tak, aby inkrementacja licznika nastąpiła (ciało owijane jest w blok, z którego `continue` wychodzi do inkrementacji).

4.4 Uściślenia jako zobowiązania

Każde miejsce, w którym wartość typu kształtu τ trafia w pozycję typu uściślonego $\{\nu : \tau \mid \varphi\}$, otrzymuje zobowiązanie. Pozycje te to:

Pozycja	**Wstawiony węzeł**

`let x: τ where φ = e`	let $x = \mathcal{E}(e)$ in (assert$[\kappa_{\text{ref}}]\varphi[x/\nu]$; .
przypisanie do `var m: τ where φ`	$m := e'$; assert$[\kappa_{\text{ref}}]\varphi[!m/\nu]$ — niezmiennik zmiennej
argument dla parametru uściślonego	włączany do warunku wstępnego wywoływanej funkcji (§6.4)
wynik funkcji o typie uściślonym	włączany do warunku końcowego funkcji
konstrukcja $C(\overline{v})$ typu T (z `where φ_T` lub o polach uściślonych $f_i : \{\nu : \tau_i \mid \psi_i\}$)	let $y = C(\overline{v})$ in $\big($assert$[\kappa_{\text{inv}}]\big(\bigwedge_i \psi_i[$ $\varphi_T[y/\text{ self}]; y\big)$ — jedno zobowiązanie `data-inv`
zapis elementu `Array⟨τ where φ⟩`	zobowiązanie φ dla zapisywanej wartości

Symetrycznie, każda zmienna typu uściślonego jest w warunkach weryfikacji **hipotezą**: przy wiązaniu zmiennej $x : \{\nu : \tau \mid \varphi\}$ dodaje się do kontekstu logicznego fakt $\varphi[x/\nu]$ (§6.3). Dla tablic o elementach uściślonych hipotezą jest $\forall k \in [0, \text{len}(a)).\varphi[a[k]/\nu]$.

4.5 Kontrakty, przykłady i właściwości

- `requires P` i `ensures Q` stają się składowymi P, Q deklaracji jądra (koniunkcja wielu klauzul).
- Blok `examples { e₁; …; eₙ }` tworzy zobowiązania κ_{ex} rozstrzygane **obliczeniem** w czasie kompilacji: każde e_i : Bool jest wartościowane przez interpreter jądra z budżetem kroków; wynik `false` to błąd `E4300` z wartościami podwyrażeń, przekroczenie budżetu to `W4301`. Przykłady nie mają dostępu do kapabilności.
- `forall x̄: e` w `examples` jest **testem właściwości**, a nie dowodem: kompilator generuje deterministycznie (ziarno = skrót tekstu funkcji) $N = 256$ wartości wejściowych spełniających uściślenia typów $\overline{x}$ i wartościuje e; kontrprzykład jest minimalizowany (shrinking) i raportowany jako `E4310`.

4.6 Domknięcia

Wyrażenie `fn(x̄: τ̄) => e` elaboruje się do domknięcia zamkniętego. Niech $\overline{y}$ będą wolnymi zmiennymi niemutowalnymi, a $\overline{m}$ wolnymi zmiennymi mutowalnymi ciała. Wtedy

$$\mathcal{E}(\text{fn}(\overline{x}) \Rightarrow e) = \text{let } \overline{z} = \overline{!m} \text{ in } (\text{fn}(\overline{x}) \Rightarrow \mathcal{E}(e)[\overline{z}/\overline{m}])$$

czyli zmienne mutowalne są **migawkowane** w chwili utworzenia domknięcia (przechwycenie przez wartość). Zapis do przechwyconej zmiennej mutowalnej w ciele domknięcia jest błędem `E6010` (Z5: lokacja nie może uciec z zasięgu). Wolne zmienne niemutowalne są po prostu podstawiane w czasie wykonania (wartości są zamknięte).

Ciało domknięcia jest wyrażeniem bez efektów sterowania i stanu: nie zawiera pętli, `return`, `break`, `continue`, przypisań, argumentów `inout` ani zagnieżdżonych domknięć (`E3015`). Obliczenie wymagające tych konstrukcji zapisuje się jako funkcję nazwaną.

Typ oczekiwany. Gdy domknięcie stoi w pozycji o znanym typie funkcyjnym T_e (argument wywołania, `let` z adnotacją), typem domknięcia jest T_e: pominięte typy parametrów są z niego przejmowane, a jawna adnotacja o innym uściśleniu parametru lub wyniku jest błędem `E2141`. Uściślenie wyniku T_e jest więc zobowiązaniem ciała domknięcia (§6.4), a nie miejsca przekazania.

Funkcje nazwane jako wartości. Nazwa f funkcji globalnej w pozycji wartości oznacza wartość typu $(\overline{x} : \overline{T}) \to T_r$ z sygnatury f. Dopuszczalne są tylko funkcje totalne, niegeneryczne, bez `requires`, bez parametrów `inout` i funkcyjnych, niebędące lematami i nie użyte jako wartość we własnej silnie spójnej składowej grafu wywołań (`E3016`). Jej znaczenie logiczne to kontrakt: $\forall \overline{x}. H(\overline{x} : \overline{T}) \Rightarrow Q_f[f(\overline{x})/\ \mathrm{result}]$.

Wartości funkcyjne nie mają równości (`E2031`). W wersji 0.5 typ funkcyjny może wystąpić tylko jako typ parametru, wiązania `let` i parametru `spec fn` — nie jako typ wyniku, argument typu (tablice, typy danych), pole, zmienna `var`, parametr `inout` ani wewnątrz innego typu funkcyjnego (`E2015`). Wartości funkcyjne nie uciekają więc z wywołania, w którym je przekazano.

4.7 Aktorzy

Deklaracja `actor a(x̄: τ) receives M { b }` elaboruje się do funkcji

$$a(\overline{x} : \overline{\tau}, \mathrm{inout\ mb} : \mathrm{Mailbox}\langle M \rangle) \to \mathrm{Unit}\ !\{\mathrm{Div}\} \cup \varepsilon_b$$

a wyrażenie `spawn a(ē)` do węzła jądra spawn $a(\overline{e})$ o typie $\mathrm{Chan}\langle M \rangle$ i efekcie $\{\mathrm{Spawn}\}$; jego kapabilność jest wymagana jako pierwszy argument (`spawn a(sp, …)`, gdzie `sp : Cap⟨Spawn⟩`).

5 Statyczna semantyka

Statyczna semantyka określa, które programy jądra są **dobrze typowane**. Operuje na typach kształtu (uściślenia są wymazywane funkcją erase); poprawność względem uściśleń i kontraktów należy do semantyki weryfikacyjnej.

5.1 Konteksty i osądy

$$\Gamma ::= \cdot \mid \Gamma, x : \tau \mid \Gamma, m : \mathrm{mut}\ \tau \mid \Gamma, \mathrm{ret} : \tau \mid \Gamma, \mathrm{loop}$$

Wpis $\mathrm{ret} : \tau$ określa typ wyniku bieżącej funkcji, wpis loop — że jesteśmy wewnątrz ciała pętli. $\Gamma|_{\mathrm{imm}}$ oznacza Γ z usuniętymi wpisami $m : \mathrm{mut}\ \tau$, ret i loop. Λ przypisuje lokacjom typy ($\Lambda(\ell) = \tau$) i jest pusty dla programów źródłowych.

Główny osąd ma postać

$$\Delta; \Gamma; \Lambda \vdash e : \tau ! \varepsilon$$

i czytamy go: *w kontekście Γ wyrażenie e ma typ τ, a jego obliczenie może wywołać efekty co najwyżej z ε*. W regułach pomijamy Δ i Λ, jeśli nie ulegają zmianie. Pozostałe osądy:

- $\Delta \vdash \tau\ \mathrm{wf}$ — poprawność typu (zmienne typowe w Δ, liczba argumentów zgodna z deklaracją);
- $\Gamma \underset{F}{\vdash} \varphi$ — formuła φ jest poprawnie posortowana i należy do fragmentu F (§6.1);
- $\vdash p : \tau \rhd \Gamma'$ — wzorzec p dopasowuje się do typu τ i wiąże zmienne Γ';
- $\vdash d\ \mathrm{ok}$ — deklaracja globalna jest poprawna.

5.2 Reguły typowania wyrażeń

5.2.1 Zmienne, odczyt i zapis

$$\frac{x : \tau \in \Gamma}{\Gamma \vdash x : \tau ! \varnothing}\ \textsc{T-Var} \qquad \frac{m : \mathrm{mut}\ \tau \in \Gamma}{\Gamma \vdash\ !m : \tau ! \varnothing}\ \textsc{T-Read} \qquad \frac{\Lambda(\ell) = \tau}{\Gamma; \Lambda \vdash\ !\ell : \tau ! \varnothing}\ \textsc{T-ReadLoc}$$

$$\frac{m : \mathrm{mut}\ \tau \in \Gamma \qquad \Gamma \vdash e : \tau ! \varepsilon}{\Gamma \vdash m := e : \mathrm{Unit}\ !\varepsilon}\ \textsc{T-Write} \qquad \frac{m : \mathrm{mut}\ \mathrm{Array}\langle \tau \rangle \in \Gamma \qquad \Gamma \vdash e_1 : \mathrm{Int}\ !\varepsilon_1 \qquad \Gamma \vdash e_2 : \tau ! \varepsilon_2}{\Gamma \vdash m[e_1] := e_2 : \mathrm{Unit}\ !\varepsilon_1 \cup \varepsilon_2}$$

T-WriteEl

Reguły dla lokacji ($\ell := e$, $\ell[e_1] := e_2$) są analogiczne z $\Lambda(\ell)$ w miejsce Γ.

5.2.2 Wiązania

$$\frac{\Gamma \vdash e_1 : \tau_1 ! \varepsilon_1 \qquad \Gamma, x : \tau_1 \vdash e_2 : \tau_2 ! \varepsilon_2}{\Gamma \vdash \text{let } x = e_1 \text{ in } e_2 : \tau_2 ! \varepsilon_1 \cup \varepsilon_2} \text{ T-Let}$$

$$\frac{\Gamma \vdash e_1 : \tau_1 ! \varepsilon_1 \qquad \Gamma, m : \text{mut } \tau_1 \vdash e_2 : \tau_2 ! \varepsilon_2}{\Gamma \vdash \text{var } m : \tau_1 = e_1 \text{ in } e_2 : \tau_2 ! \varepsilon_1 \cup \varepsilon_2} \text{ T-VarDecl}$$

Typ τ_2 jest typem wartości, więc m nie może opuścić zasięgu: lokacje nie są wartościami, a domknięcia nie mają wolnych zmiennych mutowalnych (T-Lam).

5.2.3 Wartości złożone i operacje pierwotne

$$\frac{}{\Gamma \vdash n : \text{Int } ! \varnothing} \text{ T-Lit} \qquad \frac{C(\overline{\sigma}) \in T\langle \overline{\alpha} \rangle \qquad \Gamma \vdash e_i : \sigma_i[\overline{\tau}/\overline{\alpha}] ! \varepsilon_i}{\Gamma \vdash C\langle \overline{\tau} \rangle(\overline{e}) : T\langle \overline{\tau} \rangle ! \bigcup_i \varepsilon_i} \text{ T-Ctor}$$

$$\frac{\Gamma \vdash e_i : \tau ! \varepsilon_i}{\Gamma \vdash [\overline{e}] : \text{Array}\langle \tau \rangle ! \bigcup_i \varepsilon_i} \text{ T-Arr} \qquad \frac{\text{op} : (\overline{\tau}) \xrightarrow{\varepsilon_{\text{op}}} \tau \qquad \Gamma \vdash e_i : \tau_i ! \varepsilon_i}{\Gamma \vdash \text{op}(\overline{e}) : \tau ! \varepsilon_{\text{op}} \cup \bigcup_i \varepsilon_i} \text{ T-Prim}$$

Analogiczne reguły dla literałów $\text{true}, \text{false} : \text{Bool}$, $() : \text{Unit}$, literałów bajtowych, zmiennoprzecinkowych i napisowych pomijamy. Operacje pierwotne z parametrem `inout` (np. `push`) typuje się regułą T-Call z sygnaturą z dodatku B.

5.2.4 Sterowanie

$$\frac{\Gamma \vdash e : \text{Bool } ! \varepsilon \qquad \Gamma \vdash e_1 : \tau ! \varepsilon_1 \qquad \Gamma \vdash e_2 : \tau ! \varepsilon_2}{\Gamma \vdash \text{if } e \text{ then } e_1 \text{ else } e_2 : \tau ! \varepsilon \cup \varepsilon_1 \cup \varepsilon_2} \text{ T-If}$$

$$\frac{\Gamma \vdash e : \tau ! \varepsilon_0 \qquad \vdash p_i : \tau \rhd \Gamma_i \qquad \Gamma, \Gamma_i \vdash g_i : \text{Bool } ! \varnothing \qquad \Gamma, \Gamma_i \vdash e_i' : \sigma ! \varepsilon_i \qquad \text{exhaustive}(\overline{p}, \overline{g}, \tau)}{\Gamma \vdash \text{match } e \{ \overline{p \text{ when } g \Rightarrow e'} \} : \sigma ! \varepsilon_0 \cup \bigcup_i \varepsilon_i} \text{ T-Match}$$

Strażnik g_i musi być czysty. Brak strażnika oznacza $g_i = \text{true}$.

$$\frac{\Gamma \vdash c : \text{Bool } ! \varnothing \qquad \Gamma \underset{F}{\vdash} \varphi_i \qquad \Gamma \underset{F}{\vdash} t : \text{Int} \qquad \Gamma, \text{loop} \vdash s : \text{Unit } ! \varepsilon}{\Gamma \vdash \text{while } c \text{ inv } \overline{\varphi} \text{ dec } t \text{ do } s : \text{Unit } ! \varepsilon} \text{ T-While}$$

$$\frac{\Gamma \vdash c : \text{Bool } ! \varnothing \qquad \Gamma \underset{F}{\vdash} \varphi_i \qquad \Gamma, \text{loop} \vdash s : \text{Unit } ! \varepsilon}{\Gamma \vdash \text{while } c \text{ inv } \overline{\varphi} \text{ dec } \bot \text{ do } s : \text{Unit } ! \varepsilon \cup \{\text{Div}\}} \text{ T-WhileDiv}$$

$$\frac{\text{loop} \in \Gamma}{\Gamma \vdash \text{break} : \tau ! \varnothing} \text{ T-Break} \qquad \frac{\text{loop} \in \Gamma}{\Gamma \vdash \text{continue} : \tau ! \varnothing} \text{ T-Cont} \qquad \frac{\text{ret} : \tau \in \Gamma \qquad \Gamma \vdash e : \tau ! \varepsilon}{\Gamma \vdash \text{return } e : \sigma ! \varepsilon} \text{ T-Ret}$$

Wyrażenia `break`, `continue`, `return` i `trap` mają dowolny typ, bo nie zwracają sterowania do kontekstu. Wpis loop jest usuwany przy wejściu do ciała domknięcia, więc `break` nie może przekroczyć granicy funkcji.

5.2.5 Funkcje i wywołania

$$\frac{\Gamma|_{\text{imm}}, \overline{x} : \overline{\tau}, \text{ret} : \sigma \vdash e : \sigma ! \varepsilon}{\Gamma \vdash \text{fn}(\overline{x} : \overline{\tau}) \Rightarrow e : (\overline{\tau}) \xrightarrow{\varepsilon} \sigma ! \varnothing} \text{ T-Lam}$$

Ponieważ $\Gamma|_{\text{imm}}$ nie zawiera zmiennych mutowalnych, ciało domknięcia nie może ich odczytywać ani zapisywać. Elaboracja (§4.6) migawkuje je wcześniej.

$$\frac{\Gamma \vdash e : (\overline{\tau}) \xrightarrow{\varepsilon} \sigma ! \varepsilon_0 \qquad \Gamma \vdash e'_i : \tau_i ! \varepsilon_i}{\Gamma \vdash e\big(\overline{e'}\big) : \sigma ! \varepsilon_0 \cup \bigcup_i \varepsilon_i \cup \varepsilon} \text{ T-App}$$

Dla wywołania funkcji globalnej niech $\Sigma(f) = \forall \overline{\alpha}, \overline{\rho}.(\overline{x} : \overline{\tau}, \text{inout } \overline{m} : \overline{\sigma}) \to \tau_r ! \varepsilon_f$ oraz $\theta = \big[\overline{\tau'}/\overline{\alpha}, \overline{\varepsilon'}/\overline{\rho}\big]$.

$$\frac{\Gamma \vdash e_i : \theta\tau_i ! \varepsilon_i \qquad \mu_j : \text{mut } \theta\sigma_j \in \Gamma \cup \Lambda \qquad i \neq j \Rightarrow \mu_i \neq \mu_j \qquad \tau'_k \vdash \text{bounds}(\alpha_k)}{\Gamma \vdash f\langle \overline{\tau'}, \overline{\varepsilon'} \rangle \big(\overline{e}, \overline{\&\mu}\big) : \theta\tau_r ! \bigcup_i \varepsilon_i \cup \theta\varepsilon_f} \text{ T-Call}$$

Przesłanka $i \neq j \Rightarrow \mu_i \neq \mu_j$ to **prawo wyłączności** (§5.4). Ograniczenia `Eq` i `Ord` parametrów typowych sprawdza osąd $\tau \vdash \text{bounds}(\alpha)$ (§5.5).

5.2.6 Zobowiązania, pułapki, dziury

$$\frac{\Gamma \underset{F}{\vdash} \varphi}{\Gamma \vdash \text{assert}[\kappa]\varphi : \text{Unit } ! \varnothing} \text{ T-Assert} \qquad \frac{\Gamma \underset{F}{\vdash} \varphi}{\Gamma \vdash \text{check}[\kappa]\varphi : \text{Unit } ! \varnothing} \text{ T-Check} \qquad \frac{}{\Gamma \vdash \text{trap}(k) : \tau ! \varnothing} \text{ T-Trap}$$

$$\frac{}{\Gamma \vdash \text{hole}[h] : \tau ! \varepsilon} \text{ T-Hole}$$

$$\frac{\Gamma \underset{F}{\vdash} t : \text{Int} \qquad \Gamma, t_0 : \text{Int} \vdash e : \tau ! \varepsilon}{\Gamma \vdash \text{snap}[\kappa] t_0 := t \text{ in } e : \tau ! \varepsilon} \text{ T-Snap}$$

Program zawierający hole$[h]$ jest dobrze typowany, ale **niekompletny**: kompilator raportuje dla każdej dziury jej oczekiwany typ i kontekst (§9.5) i nie generuje kodu wykonywalnego.

5.2.7 Aktorzy

$$\frac{\Sigma(a) = \text{actor}(\overline{\tau}) \text{ receives } M \qquad \Gamma \vdash e_0 : \text{Cap}\langle \text{Spawn} \rangle ! \varepsilon_0 \qquad \Gamma \vdash e_i : \tau_i ! \varepsilon_i}{\Gamma \vdash \text{spawn } a(e_0, \overline{e}) : \text{Chan}\langle M \rangle ! \{\text{Spawn}\} \cup \bigcup_i \varepsilon_i} \text{ T-Spawn}$$

$$\frac{\Gamma \vdash e_1 : \text{Chan}\langle M \rangle ! \varepsilon_1 \qquad \Gamma \vdash e_2 : M ! \varepsilon_2}{\Gamma \vdash \text{send}(e_1, e_2) : \text{Unit } ! \varepsilon_1 \cup \varepsilon_2} \text{ T-Send} \qquad \frac{\mu : \text{mut Mailbox}\langle M \rangle \in \Gamma \cup \Lambda}{\Gamma \vdash \text{recv}(\mu) : M ! \{\text{Div}\}} \text{ T-Recv}$$

Odbiór ma efekt Div, bo może blokować bez końca.

5.2.8 Osłabienie efektu

$$\frac{\Gamma \vdash e : \tau ! \varepsilon \qquad \varepsilon \subseteq \varepsilon'}{\Gamma \vdash e : \tau ! \varepsilon'} \text{ T-Sub}$$

Zawieranie efektów $\varepsilon \subseteq \varepsilon'$ rozstrzyga się syntaktycznie modulo łączność, przemienność i idempotentność $\cup$ (zmienne efektowe traktowane jak stałe).

5.3 Deklaracje i totalność

Definicja 5.1 (poprawność funkcji)**.** Deklaracja $f\langle \overline{\alpha}, \overline{\rho} \rangle (\overline{x} : \overline{\tau}, \text{inout } \overline{m} : \overline{\sigma}) \to \tau_r ! \varepsilon_f$ requires P ensures Q decreases $t = e$ jest poprawna ($\vdash f$ ok), gdy:

1. $\overline{\alpha}, \overline{\rho}; \overline{x} : \overline{\tau}, \overline{m} : \text{mut } \overline{\sigma}, \text{ret} : \tau_r \vdash e : \tau_r ! \varepsilon'$ oraz $\varepsilon' \subseteq \varepsilon_f$;
2. $\overline{x} : \overline{\tau}, \overline{m} : \text{mut } \overline{\sigma} \underset{F}{\vdash} P$ oraz $\overline{x} : \overline{\tau}, \overline{m} : \text{mut } \overline{\sigma}, \text{old}(\overline{m}) : \overline{\sigma}, \text{result} : \tau_r \underset{F}{\vdash} Q$;
3. jeśli f leży w nietrywialnej silnie spójnej składowej grafu wywołań (lub wywołuje sama siebie) i Div $\notin \varepsilon_f$, to $t \neq \bot$ oraz $\underset{F}{\vdash} t : \text{Int}$ albo $\underset{F}{\vdash} t : T\langle \overline{\tau} \rangle$ (miara strukturalna, §6.4);
4. jeśli f jest zadeklarowana bez słowa `partial`, to Div $\notin \varepsilon_f$.

Efekt ε_f funkcji zadeklarowanej w składni konkretnej jest **wyznaczany**: jest to najmniejszy zbiór spełniający punkt 1, powiększony o Div dla funkcji `partial`. Wyznaczony efekt jest jawnie wypisywany przez narzędzia

(np. w podpowiedziach i dokumentacji), a dla funkcji `pub` musi zgadzać się z deklaracją `!` w typie, jeśli taka występuje.

Z punktu 4 i reguły T-Sub wynika, że **funkcja totalna nie może wywołać funkcji częściowej**: efekt Div wywołanej funkcji musiałby zawierać się w ε_f.

Lematy. Deklaracja `lemma fn` jest funkcją totalną bez parametrów `inout`, o typie wyniku `Unit` i efekcie $\varnothing$ (błąd `E3014` w przeciwnym razie). Jej warunek końcowy jest twierdzeniem dostępnym w miejscu wywołania (reguła wywołania z §6.4); rekurencyjne wywołania w ciele lematu pełnią rolę hipotezy indukcyjnej. Przekształcenie $\mathcal{R}$ (§6.6) usuwa wywołania lematów i ich definicje z programu wykonywalnego; jest to poprawne, bo lemat jest totalny i nie ma efektów, pod warunkiem że wszystkie jego zobowiązania mają status `proved` — dlatego zobowiązanie lematu o statusie innym niż `proved` jest błędem kompilacji, a nie sprawdzeniem dynamicznym.

5.4 Prawo wyłączności

Definicja 5.2 (prawo wyłączności)**.** W każdym wywołaniu $f(\overline{e}, \&\mu_1, \ldots, \&\mu_k)$ cele $\mu_1, \ldots, \mu_k$ są parami różne. Argumentem `inout` może być wyłącznie cała zmienna mutowalna (nie element tablicy ani pole).

Ponieważ argumenty wartościowe $\overline{e}$ są obliczane **przed** wywołaniem do wartości (kopii), wystąpienie μ_j w $\overline{e}$ nie narusza wyłączności. Przekazywanie elementu tablicy jako `inout` jest *poza wersją 0.1*; w wersji 0.1 wyraża się je przez odczyt, wywołanie i zapis.

5.5 Polimorfizm, ograniczenia i inferencja

- Parametry typowe deklaruje się jawnie; argumenty typowe wywołań są **wnioskowane** przez unifikację pierwszego rzędu typów argumentów z sygnaturą (bez uogólniania `let`).
- Ograniczenie `Eq` spełniają: `Int`, `Bool`, `Byte`, `Unit`, `Str`, typy danych i tablice o argumentach `Eq`. Nie spełniają: `F64` (z powodu NaN — porównanie zapewnia `f64_eq`), typy funkcyjne, `Cap`, `Chan`, `Mailbox`.
- Ograniczenie `Ord` spełniają: `Int`, `Byte`, `Bool`, `Str` (porządek leksykograficzny bajtów).
- Zmienne efektowe ρ instancjonuje się przez dopasowanie typów funkcyjnych argumentów.

Lemat 5.3 (rozstrzygalność typowania)**.** Dla programów, w których każda deklaracja globalna ma pełne adnotacje typów parametrów i wyniku, relacja $\vdash P$ ok jest rozstrzygalna w czasie wielomianowym względem rozmiaru programu, z wyjątkiem sprawdzania wyczerpywalności (§5.6), które jest wykładnicze w najgorszym przypadku i liniowe w praktyce.

5.6 Wyczerpywalność dopasowań

Niech $\Pi = (p_i)_{i \in I}$, gdzie $I = \{i \mid g_i = \text{true}\}$, będzie macierzą wzorców gałęzi **bez strażników**. Dopasowanie jest **wyczerpujące**, $\text{exhaustive}(\overline{p}, \overline{g}, \tau)$, gdy wektor $(_)$ nie jest użyteczny względem Π w sensie algorytmu użyteczności Maranget (*Warnings for pattern matching*, JFP 2007). Gałąź ze strażnikiem nie liczy się do pokrycia.

Gdy dopasowanie nie jest wyczerpujące, algorytm wyznacza **świadka** — wzorzec wartości nieobjętej żadną gałęzią — który jest częścią diagnostyki `E5001`. Gałąź nieużyteczna względem gałęzi poprzednich daje ostrzeżenie `W5002`.

5.7 Kapabilności i brak władzy otoczenia

Wartości $\text{cap}\langle K \rangle$ nie mają nazwy w składni konkretnej i nie mogą być skonstruowane w programie. Jedynym źródłem kapabilności jest punkt wejścia

```
fn main(caps: Caps) -> Int
```

gdzie `Caps` jest typem danych preludium z polami `io`, `fs`, `net`, `clock`, `rand`, `ffi`, `spawn`.

Lemat 5.4 (władza tylko przez parametry)**.** Jeżeli $\vdash f$ ok i ciało f wywołuje efekt $K \in \text{Cap}$, to wartość $\text{cap}\langle K \rangle$ jest osiągalna z parametrów f (bezpośrednio lub przez wolne zmienne domknięcia przekazanego jako parametr).

Wynika stąd, że sygnatura funkcji (typy parametrów) jest górnym ograniczeniem jej władzy — co jest podstawą bezpieczeństwa w modelu kapabilności obiektowych.

6 Semantyka weryfikacyjna

Rozdział definiuje: logikę specyfikacji (§6.1–6.3), warunki weryfikacji i rodzaje zobowiązań (§6.4–6.5), rozstrzyganie zobowiązań i przekształcenie programu do postaci wykonywalnej (§6.6) oraz zaufaną bazę (§6.7).

6.1 Fragment logiki F

Sorty: `Int` (liczby całkowite matematyczne, bez ograniczeń), `Bool`, sorty danych algebraicznych $T\langle\bar{s}\rangle$, sort tablic $\mathrm{Array}\langle s\rangle$ modelowany jako para (długość len : Int, funkcja selekcji $\mathrm{Int} \to s$), `Byte` jako `Int` z ograniczeniem $[0, 256)$, `Str` jako $\mathrm{Array}\langle\mathrm{Byte}\rangle$. Sort `F64` oraz sorty funkcyjne i kapabilności **nie występują** w formułach (błąd `E6101`).

Termy i formuły:

```
t  ::=  x | m | old(m) | result | n | t + t | t − t | n · t | t · t
     |  t ÷ n | t mod n | len(t) | t[t] | C(t̄) | t.f | if φ then t else t | s(t̄)
φ  ::=  true | false | t ⋈ t | t is C | p(t̄)
     |  ¬φ | φ ∧ φ | φ ∨ φ | φ ⇒ φ
     |  ∀k ∈ [t, t). φ | ∃k ∈ [t, t). φ | match t { C(x̄) ⇒ φ, … }
⋈  ::=  = | ≠ | < | ≤ | > | ≥
```

gdzie s, p to funkcje i predykaty zadeklarowane przez `spec fn`. W formule zmienna mutowalna m oznacza jej **bieżącą** wartość; $\mathrm{old}(m)$ — wartość przy wejściu do funkcji (tylko w `ensures` i niezmiennikach pętli); result — wynik (tylko w `ensures`). Zapis konkretny `forall k in a..b: φ` oznacza $\forall k \in [a, b).\varphi$.

Funkcje specyfikacyjne są czyste i totalne; typ wyniku jest dowolnym typem dozwolonym w formułach (także uściślonym). Funkcja specyfikacyjna nierekurencyjna jest w warunkach weryfikacji **rozwijana** (makro logiczne). Funkcja należąca do nietrywialnej silnie spójnej składowej grafu wywołań funkcji specyfikacyjnych (**rekurencyjna**, także wzajemnie) musi mieć `decreases` (błąd `E3013`) i jest w logice **funkcją definiowaną rekurencyjnie**: symbolem s z aksjomatem $\forall\overline{x}.s(\overline{x}) = b_s(\overline{x})$, który procedura decyzyjna rozwija leniwie. Dla każdej takiej funkcji powstają zobowiązania (błąd przy statusie innym niż `proved`, bo funkcje specyfikacyjne nie mają sprawdzeń dynamicznych):

- `wd` — ciało jest określone dla każdego argumentu spełniającego uściślenia parametrów;
- `term` (`E4043`) — w każdym wywołaniu w obrębie składowej, przy warunkach ścieżki (gałęzie `match` i `if`, lewy argument `&&`/`||`, zakres kwantyfikatora), miara argumentów jest nieujemna i mniejsza niż miara parametrów;
- `post` — jeśli typ wyniku jest uściślony, ciało spełnia uściślenie przy **hipotezie indukcyjnej** $\forall\overline{y}.\mu(\overline{y}) < \mu(\overline{x}) \wedge H(\overline{y}) \Rightarrow h(s(\overline{y}), \tau_r)$ dla wszystkich funkcji składowej.

Po udowodnieniu uściślenie wyniku jest dodawane do pozostałych warunków weryfikacji jako aksjomat $\forall\overline{x}.H(\overline{x}) \Rightarrow h(s(\overline{x}), \tau_r)$ (wzorzec: $s(\overline{x})$). Poprawność wynika z indukcji dobrze ufundowanej po mierze (zobowiązania `term`).

Definicja 6.1 (fragment rozstrzygalny F_0)**.** $F_0 \subset F$ to formuły bez mnożenia zmiennych przez zmienne, bez rekurencyjnych funkcji specyfikacyjnych, w których kwantyfikatory po zanegowaniu i skolemizacji tworzą formułę *fragmentu własności tablic* (Bradley, Manna, Sipma, *What's decidable about arrays?*, VMCAI 2006) z teorią elementów będącą liniową arytmetyką całkowitoliczbową.

Twierdzenie 6.2 (rozstrzygalność F_0)**.** Prawdziwość formuł F_0 jest rozstrzygalna.

Szkic dowodu. Bezpośrednio z rozstrzygalności fragmentu własności tablic nad LIA [BMS06] po rozwinięciu nierekurencyjnych funkcji specyfikacyjnych (rozwinięcie zachowuje fragment, bo ciała funkcji specyfikacyjnych należą do F_0 z definicji). □

Hipoteza 6.3. Fragment F_0 rozszerzony o teorię skończonych typów danych algebraicznych (konstruktory, testery, selektory) pozostaje rozstrzygalny. Uzasadnienie: kombinacja Nelsona–Oppena teorii rozłącznych sygnatur z teorią ADT (Oppen 1980), przy czym wymaga sprawdzenia warunku stabilnej nieskończoności dla sortów elementów. Status: do dowodu.

Rozstrzygalność nie jest warunkiem poprawności: dla formuł spoza F_0 procedura może zwrócić *nieznane*, co prowadzi do sprawdzenia w czasie wykonania (§6.6), a nie do utraty poprawności.

6.2 Dobra określoność formuł

Termy $t[u]$ i $t.f$ są częściowe. Każda formuła z kontraktu, niezmiennika lub uściślenia musi być **dobrze określona** w miejscu użycia. Warunek dobrej określoności wd jest zdefiniowany z obliczaniem leniwym od lewej do prawej:

$$\begin{aligned}
\mathrm{wd}(t[u]) &= \mathrm{wd}(t) \wedge \mathrm{wd}(u) \wedge 0 \le u < \mathrm{len}(t) \\
\mathrm{wd}(t.f_C) &= \mathrm{wd}(t) \wedge t \text{ is } C \\
\mathrm{wd}(\varphi \wedge \psi) &= \mathrm{wd}(\varphi) \wedge (\varphi \Rightarrow \mathrm{wd}(\psi)) \\
\mathrm{wd}(\varphi \vee \psi) &= \mathrm{wd}(\varphi) \wedge (\neg\varphi \Rightarrow \mathrm{wd}(\psi)) \\
\mathrm{wd}(\varphi \Rightarrow \psi) &= \mathrm{wd}(\varphi) \wedge (\varphi \Rightarrow \mathrm{wd}(\psi)) \\
\mathrm{wd}(\forall k \in [a,b).\varphi) &= \mathrm{wd}(a) \wedge \mathrm{wd}(b) \wedge \forall k \in [a,b).\,\mathrm{wd}(\varphi) \\
\mathrm{wd}(\text{if } \varphi \text{ then } t_1 \text{ else } t_2) &= \mathrm{wd}(\varphi) \wedge (\varphi \Rightarrow \mathrm{wd}(t_1)) \wedge (\neg\varphi \Rightarrow \mathrm{wd}(t_2))
\end{aligned}$$

i homomorficznie w pozostałych przypadkach. Dla każdej formuły φ użytkownika powstaje zobowiązanie rodzaju `wd`: $\mathrm{wd}(\varphi)$ ma zachodzić przy hipotezach obowiązujących w miejscu jej użycia (dla `ensures` — przy P i hipotezach typów; dla niezmiennika — przy poprzednich klauzulach niezmiennika).

6.3 Hipotezy typów

Dla kontekstu Γ definiujemy formułę hipotez

$$H(\Gamma) = \bigwedge_{x:\tau \in \Gamma} h(x,\tau) \quad \text{gdzie} \quad h(x,\{\nu : \tau \mid \varphi\}) = \varphi[x/\nu] \wedge h(x,\tau)$$

oraz dla typów kształtu: $h(x, T\langle\ldots\rangle) = \varphi_{T[x/\ \mathrm{self}]}$ (niezmiennik typu danych), $h\big(f, (\overline{y} : \overline{T}) \to \{\nu : \sigma \mid \psi\}\big) = \forall \overline{y}.H\big(\overline{y} : \overline{T}\big) \Rightarrow \psi[f(\overline{y})/\nu]$ (typ funkcyjny), $h(x, \mathrm{Array}\langle\sigma\rangle) = 0 \le \mathrm{len}(x) \le \mathrm{MAX} \wedge \forall k \in [0,\mathrm{len}(x)).h(x[k],\sigma)$, $h(x,\mathrm{Int}) = \mathrm{MIN} \le x \le \mathrm{MAX}$, $h(x,\mathrm{Byte}) = 0 \le x < 256$ i $\top$ w pozostałych przypadkach. Dla typu danych o argumentach uściślonych, np. $\mathrm{Option}\langle\{\nu : \mathrm{Int} \mid \nu \ge 0\}\rangle$, h zawiera ponadto warunki na polach: $\mathrm{match}\ x \big\{ C(\overline{y}) \Rightarrow \bigwedge_j h(y_j,\tau_j) \big\}$ dla każdego konstruktora C. Stałe $\mathrm{MIN} = -2^{63}$, $\mathrm{MAX} = 2^{63} - 1$ są w programach dostępne jako `int_min` i `int_max`.

6.4 Warunki weryfikacji

6.4.1 Zobowiązania i fokus

Każde zobowiązanie κ jest weryfikowane **osobno**, przy założeniu wszystkich pozostałych (weryfikacja „assume-after-assert"). Dla ustalonego zobowiązania w fokusie κ^* definiujemy

$$\mathrm{ob}_\kappa(\varphi, X) = \begin{cases} \varphi \wedge X & \text{gdy } \kappa = \kappa^* \\ \varphi \Rightarrow X & \text{gdy } \kappa \ne \kappa^* \end{cases}$$

Poprawność tej konstrukcji przy zobowiązaniach nieudowodnionych gwarantuje to, że stają się one sprawdzeniami w czasie wykonania (tw. 8.6).

6.4.2 Postkondycje strukturalne

Warunek weryfikacji wyrażenia liczony jest względem czwórki $Q = \langle N, B, C, R\rangle$: N — predykat na wartości ν przy normalnym zakończeniu, B — formuła przy `break`, C — formuła przy `continue`, R — predykat na ν przy `return`. Zapis $Q[N := N']$ oznacza zamianę jednej składowej. $N(v)$ skraca $N[v/\nu]$. Dla ciągu argu-

mentów $\mathrm{wp}(\overline{e}, \lambda\overline{v}.X)$ oznacza złożenie od lewej do prawej: $\mathrm{wp}(e_1, Q[N := \lambda v_1.\, \mathrm{wp}(e_2, \ldots \mathrm{wp}(e_n, Q[N := \lambda v_n.X])\ldots)])$.

6.4.3 Reguły

Zmienne związane przez let, var i wzorce są przemianowane tak, by były unikalne (konwencja Barendregta), więc podstawienia nie przechwytują zmiennych.

$$\begin{aligned}
\mathrm{wp}(v, Q) &= N(v) \\
\mathrm{wp}(x, Q) &= N(x) \\
\mathrm{wp}(!m, Q) &= N(m) \\
\mathrm{wp}(\text{let } x = e_1 \text{ in } e_2, Q) &= \mathrm{wp}(e_1, Q[N := \lambda\nu.\, \mathrm{wp}(e_2, Q)[\nu/x]]) \\
\mathrm{wp}(\text{var } m : \tau = e_1 \text{ in } e_2, Q) &= \mathrm{wp}(e_1, Q[N := \lambda\nu.\, \mathrm{wp}(e_2, Q)[\nu/m]]) \\
\mathrm{wp}(m := e, Q) &= \mathrm{wp}(e, Q[N := \lambda\nu.N(())[\nu/m]]) \\
\mathrm{wp}(m[e_1] := e_2, Q) &= \mathrm{wp}((e_1, e_2), \lambda(i, v).\mathrm{ob}_\kappa(0 \le i < \mathrm{len}(m), N(())[m\langle i \mapsto v\rangle/m]))
\end{aligned}$$

Operacje pierwotne mają kontrakty $\big(\mathrm{pre}_{\mathrm{op}}, \mathrm{post}_{\mathrm{op}}\big)$ z dodatku B; w szczególności:

$$\begin{aligned}
\mathrm{wp}(\mathrm{add}(e_1, e_2), Q) &= \mathrm{wp}((e_1, e_2), \lambda(a, b).\mathrm{ob}_\kappa(\mathrm{MIN} \le a + b \le \mathrm{MAX}, N(a + b))) \\
\mathrm{wp}(\mathrm{div}(e_1, e_2), Q) &= \mathrm{wp}((e_1, e_2), \lambda(a, b).\mathrm{ob}_\kappa(b \ne 0 \wedge \neg(a = \mathrm{MIN} \wedge b = -1), N(a \div b))) \\
\mathrm{wp}(\mathrm{get}(e_1, e_2), Q) &= \mathrm{wp}((e_1, e_2), \lambda(a, i).\mathrm{ob}_\kappa(0 \le i < \mathrm{len}(a), N(a[i]))) \\
\mathrm{wp}(\mathrm{op}(\overline{e}), Q) &= \mathrm{wp}\big(\overline{e}, \lambda\overline{v}.\mathrm{ob}_\kappa\big(\mathrm{pre}_{\mathrm{op}}(\overline{v}), \forall r.\mathrm{post}_{\mathrm{op}}(\overline{v}, r) \Rightarrow N(r)\big)\big)
\end{aligned}$$

gdzie $\div$ oznacza dzielenie obcinane w stronę zera. Sterowanie:

$$\begin{aligned}
\mathrm{wp}(\text{if } e \text{ then } e_1 \text{ else } e_2, Q) &= \mathrm{wp}(e, Q[N := \lambda b.(b \Rightarrow \mathrm{wp}(e_1, Q)) \wedge (\neg b \Rightarrow \mathrm{wp}(e_2, Q))]) \\
\mathrm{wp}(\mathrm{break}, Q) &= B \\
\mathrm{wp}(\mathrm{continue}, Q) &= C \\
\mathrm{wp}(\mathrm{return}\ e, Q) &= \mathrm{wp}(e, Q[N := R]) \\
\mathrm{wp}(\mathrm{trap}(\mathrm{unreachable}), Q) &= \mathrm{ob}_\kappa(\mathrm{false}, \top)
\end{aligned}$$

Dopasowanie definiuje się przez rozwinięcie do kaskady testów: $\mathrm{match}\ e\{p_1\ \mathrm{when}\ g_1 \Rightarrow e_1, \rho\}$ to $\mathrm{let}\ v = e\ \mathrm{in}\ \mathrm{if}\ \mathrm{test}(p_1, v)\ \mathrm{then}\ \big(\mathrm{let}\ \overline{x} = \mathrm{sel}(p_1, v)\ \mathrm{in}\ \mathrm{if}\ g_1\ \mathrm{then}\ e_1\ \mathrm{else}\ M_\rho\big)\ \mathrm{else}\ M_\rho$, gdzie M_ρ jest rozwinięciem reszty gałęzi, a pusta reszta to $\mathrm{trap}(\mathrm{unreachable})$ (zobowiązanie rodzaju `unreach`, zawsze dowodliwe dla dopasowań wyczerpujących). Funkcje test i sel wyrażają się testerami i selektorami teorii ADT. Zmienne $\overline{x}$ wiązane wzorcem są wartościami typów swoich pól, więc do kontekstu dodaje się hipotezy $h(x_i, \tau_i)$ (§6.3) — w szczególności niezmiennik typu dla pól typu rekurencyjnego, którego nie obejmuje ograniczona głębokość hipotezy wartości dopasowywanej.

6.4.4 Pętle

Niech $I = \varphi_1 \wedge \ldots \wedge \varphi_k$ będzie niezmiennikiem (każda klauzula φ_i ma własne zobowiązania rodzajów `inv-init` i `inv-pres`), $\overline{w} = \mathrm{ws}(c) \cup \mathrm{ws}(s)$ zbiorem zmiennych mutowalnych, do których pętla może pisać (cele $:=$, zapisów elementów, argumentów `inout`), a warunek c — czysty. Dla pętli z wariantem t:

$$\mathrm{wp}(\text{while } c \text{ inv } I \text{ dec } t \text{ do } s, Q) = \mathrm{ob}_{\kappa_{\mathrm{init}}}(I, \forall\overline{w}.I \Rightarrow \mathrm{wp}(c, Q[N := \lambda b.\Phi_b]))$$

$$\Phi_b = \Big(b \Rightarrow \forall t_0.t_0 = t \Rightarrow \mathrm{ob}_{\kappa_{\mathrm{nn}}}(t_0 \ge 0, \mathrm{wp}(s, Q_s))\Big) \wedge (\neg b \Rightarrow N(()))$$

$$Q_s = \langle \lambda_.J,\ N(()),\ J,\ R\rangle, \quad J = \mathrm{ob}_{\kappa_{\mathrm{pres}}}\Big(I, \mathrm{ob}_{\kappa_{\mathrm{dec}}}(t < t_0, \top)\Big)$$

Dla pętli bez wariantu ($t = \bot$) usuwa się zobowiązania κ_{nn} i κ_{dec}. `break` prowadzi do $N(())$, czyli postkondycji normalnego zakończenia pętli; `continue` — do J.

6.4.5 Wywołania

Dla $\Sigma(f)$ z warunkami P_f, Q_f, miarą t_f i wywołania w ciele funkcji g:

$$\mathrm{wp}(f(\overline{e}, \&\overline{\mu}), Q) = \mathrm{wp}\Big(\overline{e}, \lambda\overline{v}.\mathrm{ob}_{\kappa_{\mathrm{pre}}}\big(P_f[\overline{v}/\overline{x}, \overline{\mu}/\overline{m}],\, \mathrm{ob}_{\kappa_{\mathrm{term}}}(T,\, \Psi)\big)\Big)$$

$$\Psi = \forall r, \overline{\mu'}.\; Q_f\left[\overline{v}/\overline{x}, \overline{\mu}/\,\mathrm{old}(\overline{m}), \overline{\mu'}/\overline{m}, r/\,\mathrm{result}\right] \Rightarrow N(r)\left[\overline{\mu'}/\overline{\mu}\right]$$

Obligacja terminacji $T = 0 \le t_f[\overline{v}/\overline{x}, \overline{\mu}/\overline{m}] < t_g^0$ występuje wtedy i tylko wtedy, gdy f i g należą do tej samej silnie spójnej składowej grafu wywołań oraz $\mathrm{Div} \notin \varepsilon_g$; t_g^0 to wartość miary g przy wejściu. W pozostałych przypadkach $T = \top$.

Kwantyfikacja tylko po $\overline{\mu'}$ (a nie po wszystkich zmiennych) jest poprawna dzięki **własności ramki** (lemat 8.3): wywołanie może zmienić wyłącznie lokacje przekazane jako `inout`.

Wywołanie domknięcia $e\big(\overline{e'}\big)$ o typie $\Big(\overline{\{\nu : \tau \mid \varphi\}}\Big) \xrightarrow{\varepsilon} \{\nu : \sigma \mid \psi\}$ daje obligację $\bigwedge_i \varphi_i(v_i)$ rodzaju `pre` i hipotezę $\psi(r)$ dla wyniku. Utworzenie domknięcia $\mathrm{fn}(\overline{x}) \Rightarrow e$ generuje w miejscu utworzenia koniunkcję $\forall\overline{x}.H(\overline{x} : \overline{\tau}) \Rightarrow \mathrm{wp}(e, \langle\psi, \top, \top, \psi\rangle)$, w której zobowiązania ψ mają rodzaj `refine`. Domknięcie, którego ciało należy do fragmentu F, jest w logice termem $\lambda\overline{x}.[\![e]\!]$; pozostałe są znane wyłącznie przez swój typ $h(f, T)$.

Przekazanie wartości funkcyjnej. Argument v dla parametru typu $T_d = \Big(\overline{x} : \overline{\{\nu : \tau \mid \varphi^d\}}\Big) \to \{\nu : \sigma \mid \psi^d\}$, którego wartość ma typ źródłowy $T_s = \Big(\overline{x} : \overline{\{\nu : \tau \mid \varphi^s\}}\Big) \to \{\nu : \sigma \mid \psi^s\}$, daje dwa zobowiązania rodzaju `pre`: kontrawariancję parametrów $\forall\overline{x}.\, \bigwedge_i \varphi_i^d \Rightarrow \bigwedge_i \varphi_i^s$ oraz kowariancję wyniku $h(v, T_d)$. Dla funkcji nazwanej f druga obligacja przyjmuje postać bezkwantyfikatorową $\forall\overline{x}, r.\, \bigwedge_i \varphi_i^d \wedge Q_f[r/\,\mathrm{result}] \Rightarrow \psi^d(r)$, rozstrzygalną bez aksjomatu kontraktu.

6.4.6 Zobowiązania jawne, sprawdzenia, dziury i aktorzy

$$\begin{aligned}
\mathrm{wp}(\mathrm{assert}[\kappa]\varphi, Q) &= \mathrm{ob}_\kappa(\varphi, N(())) \\
\mathrm{wp}(\mathrm{check}[\kappa]\varphi, Q) &= \varphi \Rightarrow N(()) \\
\mathrm{wp}(\mathrm{snap}[\kappa]t_0 := t \text{ in } e, Q) &= \mathrm{wp}(e, Q)[t/t_0] \\
\mathrm{wp}(\mathrm{hole}[h], Q) &= \top \\
\mathrm{wp}(\mathrm{recv}(\mu), Q) &= \forall v.h(v, M) \Rightarrow N(v) \\
\mathrm{wp}(\mathrm{send}(e_1, e_2), Q) &= \mathrm{wp}((e_1, e_2), \lambda(c, v).\mathrm{ob}_\kappa(h(v, M), N(())))
\end{aligned}$$

Dziura jest interpretowana **anielsko** ($\top$): zobowiązania na ścieżkach przez dziurę są spełnione. W zamian raport dziury (§9.5) zawiera jej **cel**: predykat N (oraz B, C, R), który musi spełniać wartość wstawiona w miejsce dziury.

6.4.7 Warunek weryfikacji funkcji

Dla f z parametrami $\overline{x} : \overline{\tau}, \overline{m} : \overline{\sigma}$, warunkami P, Q, miarą t i ciałem e oraz zobowiązania κ^* leżącego w f:

$$Q_{\mathrm{ret}} = \langle\lambda\nu.\mathrm{ob}_{\kappa_{\mathrm{post}}}(Q[\nu/\,\mathrm{result}], \top),\, \top,\, \top,\, \lambda\nu.\mathrm{ob}_{\kappa_{\mathrm{post}}}(Q[\nu/\,\mathrm{result}], \top)\rangle$$

$$\mathrm{VC}_{\kappa^*}(f) = \forall\overline{x}, \overline{m}.\; H(\overline{x} : \overline{\tau}, \overline{m} : \overline{\sigma}) \wedge P \Rightarrow \mathrm{wp}(e, Q_{\mathrm{ret}})[\overline{m}/\,\mathrm{old}(\overline{m}), t/t^0]$$

Podstawienie $[\overline{m}/\,\mathrm{old}(\overline{m})]$ utożsamia wartości początkowe parametrów `inout` z ich wartościami przy wejściu, a t^0 — wartość miary przy wejściu — z t.

Program jest **zweryfikowany**, gdy $\vDash \mathrm{VC}_\kappa(f)$ dla każdego zobowiązania κ każdej funkcji f.

6.4.8 Miary

Miara `decreases t` (funkcji, pętli lub funkcji specyfikacyjnej) jest termem typu `Int` albo typu danych $T\langle\overline{\tau}\rangle$. Dla typu danych miarą jest **ranga**:

$$\mathrm{rank}_T(C(v_1, \ldots, v_n)) = 1 + \sum_{i:\tau_i = T\langle\overline{\tau}\rangle} \mathrm{rank}_T(v_i)$$

czyli liczba węzłów wartości wzdłuż pól tego samego typu. Ranga jest dodatnia, więc warunek nieujemności jest spełniony trywialnie (procedura decyzyjna otrzymuje aksjomat $\forall x.\mathrm{rank}_T(x) \geq 1$), a warunek spadku $\mathrm{rank}(t') < \mathrm{rank}(t)$ zachodzi dla każdej ścisłej podstruktury: argument wywołania rekurencyjnego musi być polem (lub polem pola) wartości dopasowanej do parametru. Spadek rangi jest dobrze ufundowany, bo wartości są drzewami skończonymi.

6.5 Rodzaje zobowiązań

Rodzaj	Źródło	Jawne?	Kod błędu
`pre`	warunek wstępny wywoływanej funkcji (w tym uściślenia parametrów)	tak	E4010
`post`	warunek końcowy (w tym uściślenie wyniku)	tak	E4020
`inv-init`	niezmiennik pętli przy wejściu	tak	E4030
`inv-pres`	zachowanie niezmiennika przez obrót pętli	tak	E4031
`var-nn`	nieujemność wariantu	tak	E4040
`var-dec`	spadek wariantu	tak	E4041
`term`	spadek miary w wywołaniu rekurencyjnym (funkcja / funkcja specyfikacyjna)	tak	E4042 / E4043
`refine`	uściślenie przy wiązaniu lub przypisaniu	tak	E4050
`data-inv`	niezmiennik typu danych i uściślenia pól przy konstrukcji	tak	E4051
`assert`	instrukcja `assert`	tak	E4060
`wd`	dobra określoność formuły	tak	E4070
`unreach`	osiągalność końca dopasowania	tak	E4080
`bounds`	indeks w zakresie	nie	E4110
`overflow`	brak przepełnienia	nie	E4120
`div0`	dzielnik niezerowy i brak przepełnienia dzielenia	nie	E4130
`example`	przykład z bloku `examples`	tak	E4300
`property`	test właściwości `forall`	tak	E4310

Zobowiązania **jawne** wynikają ze specyfikacji napisanej przez użytkownika; **niejawne** — z semantyki operacji. Rozróżnienie wpływa na rozstrzyganie kontrprzykładów indukcyjnych (§6.6).

6.6 Rozstrzyganie zobowiązań

6.6.1 Interfejs procedury decyzyjnej

Procedura decyzyjna to funkcja $\mathrm{Solve}_\beta : \mathrm{Formuła} \to \{\mathrm{Valid}, \mathrm{Invalid}(M), \mathrm{Unknown}\}$ z budżetem $\beta \in \mathbb{N}$ mierzonym w krokach wnioskowania (nie w czasie). Implementacja **musi** spełniać:

- **(S1)** *Poprawność:* $\mathrm{Solve}_\beta(\Psi) = \mathrm{Valid} \Rightarrow \models \Psi$.
- **(S2)** *Walidowane kontrprzykłady:* $\mathrm{Solve}_\beta(\Psi) = \mathrm{Invalid}(M) \Rightarrow M \not\models \Psi$, przy czym implementacja sprawdza to bezpośrednim wartościowaniem Ψ w M (każda formuła F jest wykonywalna, tw. 8.8).
- **(S3)** *Determinizm:* wynik zależy wyłącznie od (Ψ, β) i wersji kompilatora.
- **(S4)** *Monotoniczność budżetu:* jeśli $\mathrm{Solve}_\beta(\Psi) \neq \mathrm{Unknown}$, to $\mathrm{Solve}_{\beta'}(\Psi) = \mathrm{Solve}_\beta(\Psi)$ dla $\beta' \geq \beta$.

6.6.2 Klasyfikacja kontrprzykładów

Model M zanegowanego $\mathrm{VC}_\kappa(f)$ przypisuje wartości parametrom f oraz zmiennym kwantyfikowanym wewnątrz warunku (zmienne pętli po „havoc”, wyniki wywołań). Kontrprzykład jest:

- **rzeczywisty**, gdy wykonanie f na wartościach parametrów z M (interpreter jądra, budżet kroków, bez kapabilności) kończy się naruszeniem κ — wtedy dołącza się ślad wykonania; także gdy κ leży na ścieżce bez pętli i wywołań (wtedy M jest rzeczywisty z konstrukcji);

- **indukcyjny**, w pozostałych przypadkach: stan z M spełnia hipotezy (niezmienniki, postkondycje wywołań), ale może być nieosiągalny. Diagnoza brzmi wówczas: *niezmiennik lub kontrakt wywoływanej funkcji jest za słaby.*

Gdy f wymaga kapabilności, odtworzenie nie jest wykonywane i kontrprzykład otrzymuje status `not-replayable`.

Parametry funkcyjne. Hipoteza $h(g, T)$ parametru funkcyjnego jest kwantyfikowana, więc procedura decyzyjna często zwraca Unknown. Wtedy zastępuje się ją instancjami w punktach $\overline{t}$, w których formuła stosuje g (poza zasięgiem innych kwantyfikatorów; w przeciwnym razie krok jest pomijany). Model M tak powstałej formuły jest kontrprzykładem rzeczywistym, jeśli typ T jest **punktowo zamieszkany**: $\forall \overline{y}.H\big(\overline{y} : \overline{T}\big) \Rightarrow \exists r.\psi(r)$ — wartości g z M w punktach $\overline{t}$ przedłużają się wtedy do funkcji typu T. Kontrprzykład przedstawia g jako tabelę wartości w tych punktach. Wynik wywołania domknięcia znanego wyłącznie przez typ jest abstrakcją (kontrprzykład indukcyjny); wynik wywołania parametru — nie.

6.6.3 Status i przekształcenie programu

Dla każdego zobowiązania κ z formułą φ_κ:

Wynik	Skutek
Valid	Status `proved`. Węzeł assert$[\kappa]$ jest usuwany; dla zobowiązań niejawnych sprawdzenie dynamiczne (granic, przepełnienia, dzielenia) jest **eliminowane** z kodu.
Unknown	Dla zobowiązań z parametrami funkcyjnymi najpierw instancjonowanie (§6.6.2). Następnie **próba obalenia testowaniem**: deterministycznie generowane wejścia spełniające warunki wstępne (ziarno wyznaczone z identyfikatora zobowiązania) są wykonywane w interpreterze; naruszenie tego samego zobowiązania daje status `refuted` z kontrprzykładem rzeczywistym (zminimalizowanym). W przeciwnym razie status `unknown`: węzeł zastępowany przez check$[\kappa]\varphi_\kappa$ (dla niejawnych zostaje sprawdzenie dynamiczne), ostrzeżenie `W4200` z listą „niedowiedzionych obietnic". W funkcjach specyfikacyjnych i lematach `unknown` jest błędem. Niedowiedzione zobowiązanie typu funkcyjnego przy przekazaniu wartości staje się **monitorem**: przekazywana jest funkcja opakowująca, która przy każdym wywołaniu sprawdza uściślenia parametrów funkcji źródłowej i uściślenie wyniku typu docelowego (kontrakty wyższego rzędu w stylu Findlera–Felleisena).
Invalid, rzeczywisty	Status `refuted`. Błąd kompilacji o kodzie z tabeli §6.5.
Invalid, indukcyjny, jawne	Status `refuted`. Błąd kompilacji.
Invalid, indukcyjny, niejawne	Status `refuted-inductive`. Ostrzeżenie `W4400`, sprawdzenie dynamiczne pozostaje.

Wymienione przekształcenie oznaczamy $\mathcal{R}$; usuwa ono ponadto definicje i wywołania lematów. Program wykonywalny to $\mathcal{R}(\mathcal{E}(P))$; nie zawiera on węzłów assert ani hole. Domyślny budżet β jest stałą kompilatora; zmiana budżetu może jedynie zamienić `unknown` na `proved` lub `refuted` (S4).

6.7 Zaufana baza

Gwarancje rozdziału 8 zależą od następujących założeń, które kompilator wypisuje w raporcie weryfikacji:

1. poprawność procedury decyzyjnej (S1);
2. kontrakty funkcji `extern` — przyjmowane bez dowodu; każde wywołanie `extern` ma ponadto sprawdzenie dynamiczne warunku końcowego, o ile jest on wykonywalny;
3. poprawność kompilatora z jądra do kodu docelowego (C lub WebAssembly) i środowiska wykonawczego; w prototypie oba generatory (Python i C) są nieweryfikowane i testowane różnicowo względem interpre-

tera jądra (w tym z wymuszonymi sprawdzeniami dynamicznymi i z programami, których zobowiązania obalono, oraz — dla C — pod kontrolą UBSan i ASan);
4. zgodność modelu kosztów: nieobecność przepełnienia stosu wywołań nie jest gwarantowana (przepełnienie stosu jest pułapką `stack`, traktowaną jak Div).

7 Dynamiczna semantyka

Semantyka jest zdefiniowana dla programów wykonywalnych $\mathcal{R}(\mathcal{E}(P))$ (bez węzłów assert i hole). Po przekształceniu $\mathcal{R}$ adnotacje pętli nie mają znaczenia operacyjnego: pozostałe sprawdzenia niezmienników i wariantów występują jako jawne węzły check i snap na początku obliczania warunku i na końcu ciała pętli, a odwołania do $\mathrm{old}(m)$ są zastępowane zmiennymi migawkowanymi przy wejściu do funkcji.

7.1 Konfiguracje i zdarzenia

Pamięć to skończona funkcja częściowa $\sigma : \mathrm{Loc} \rightharpoonup \mathrm{Val}$. Konfiguracja pojedynczego aktora to para $\langle e, \sigma \rangle$. Przejścia są etykietowane zdarzeniami $\alpha ::= \tau \mid K.\mathrm{op}(\overline{v})$, gdzie τ oznacza krok wewnętrzny, a $K.\mathrm{op}(\overline{v})$ — obserwowalne użycie kapabilności K (np. $\mathrm{Io.print}(s)$). Piszemy $\to$ zamiast $\xrightarrow{\tau}$.

Wartości semantyczne są **drzewami skończonymi**: gramatyka wartości (§3.2) nie zawiera lokacji, więc wartość nie może wskazywać na pamięć. Tablica w pamięci jest wartością, a zapis elementu tworzy (semantycznie) nową wartość.

7.2 Konteksty obliczeń

```
E  ::=  [ ]
     |  let x = E in e  |  var m : τ = E in e
     |  op(v̄, E, ē)  |  C⟨τ⟩(v̄, E, ē)  |  [v̄, E, ē]
     |  if E then e else e  |  match E { … }
     |  ℓ := E  |  ℓ[E] := e  |  ℓ[v] := E
     |  f⟨τ̄;ε⟩(v̄, E, ē, &)  |  E(ē)  |  v(v̄, E, ē)
     |  return E  |  send(E, e)  |  send(v, E)  |  spawn a(v̄, E, ē)
     |  ret{E}  |  brk{E}  |  cnt{E}
```

Konteksty ograniczone: E_r — bez ramek ret; E_b — bez ramek brk i ret; E_c — bez ramek cnt i ret. Wiązanie zmiennych w kontekstach jest standardowe; obliczanie przebiega od lewej do prawej.

7.3 Reguły redukcji

7.3.1 Wiązania i pamięć

$$
\begin{array}{lll}
\text{let } x = v \text{ in } e & \to e[v/x] & \text{(R-Let)} \\
\langle \text{var } m : \tau = v \text{ in } e, \sigma \rangle & \to \langle e[\ell/m], \sigma[\ell \mapsto v] \rangle \quad \ell \notin \mathrm{dom}(\sigma) & \text{(R-Var)} \\
\langle !\ell, \sigma \rangle & \to \langle \sigma(\ell), \sigma \rangle & \text{(R-Read)} \\
\langle \ell := v, \sigma \rangle & \to \langle (), \sigma[\ell \mapsto v] \rangle & \text{(R-Write)} \\
\langle \ell[n] := v, \sigma \rangle & \to \langle (), \sigma[\ell \mapsto \sigma(\ell)\langle n \mapsto v \rangle] \rangle \quad 0 \le n < |\sigma(\ell)| & \text{(R-WrEl)} \\
\langle \ell[n] := v, \sigma \rangle & \to \langle \mathrm{trap}(\mathrm{bounds}), \sigma \rangle \quad \text{w p.p.} & \text{(R-WrElT)}
\end{array}
$$

Lokacja przydzielona przez R-Var jest martwa po zakończeniu obliczania e (nie występuje w żadnej wartości ani w kontynuacji); implementacja zwalnia ją przy wyjściu z zasięgu. Semantyka nie modeluje zwalniania, bo nie ma ono obserwowalnych skutków.

7.3.2 Operacje pierwotne

$$
\begin{array}{lll}
\mathrm{add}(n_1, n_2) & \to n_1 + n_2 \quad \text{gdy } \mathrm{MIN} \le n_1 + n_2 \le \mathrm{MAX}, \text{w p.p. } \mathrm{trap}(\mathrm{overflow}) & \text{(R-Add)} \\
\mathrm{div}(n_1, 0) & \to \mathrm{trap}(\mathrm{div0}) & \text{(R-Div0)} \\
\mathrm{div}(\mathrm{MIN}, -1) & \to \mathrm{trap}(\mathrm{overflow}) & \text{(R-DivOv)} \\
\mathrm{div}(n_1, n_2) & \to n_1 \div n_2 \quad \text{w p.p.} & \text{(R-Div)} \\
\mathrm{get}([v_0, \ldots, v_{k-1}], n) & \to v_n \quad \text{gdy } 0 \le n < k, \text{w p.p. } \mathrm{trap}(\mathrm{bounds}) & \text{(R-Get)} \\
\mathrm{len}([v_0, \ldots, v_{k-1}]) & \to k & \text{(R-Len)} \\
\langle \mathrm{push}(\&\ell, v), \sigma \rangle & \to \langle (), \sigma[\ell \mapsto \sigma(\ell) \cdot [v]] \rangle & \text{(R-Push)}
\end{array}
$$

Pozostałe operacje (`sub`, `mul`, `rem`, porównania, `pop`, napisy) są zdefiniowane analogicznie w dodatku B.

7.3.3 Sterowanie

$$
\begin{array}{lll}
\text{if true then } e_1 \text{ else } e_2 & \to e_1 & \text{(R-IfT)} \\
\text{if false then } e_1 \text{ else } e_2 & \to e_2 & \text{(R-IfF)} \\
\mathrm{match}\ v\{p_1 \text{ when } g_1 \Rightarrow e_1, \rho\} & \to \text{if } g_1\theta \text{ then } e_1\theta \text{ else match } v\{\rho\} \quad \text{gdy } \mathrm{match}(p_1, v) = \theta & \text{(R-Match)} \\
\mathrm{match}\ v\{p_1 \text{ when } g_1 \Rightarrow e_1, \rho\} & \to \mathrm{match}\ v\{\rho\} \quad \text{gdy } \mathrm{match}(p_1, v) \text{ nie istnieje} & \text{(R-MatchN)} \\
\mathrm{match}\ v\{\} & \to \mathrm{trap}(\mathrm{unreachable}) & \text{(R-Match0)}
\end{array}
$$

$$
\begin{array}{lll}
\text{while } c \text{ do } s & \to \mathrm{brk}\{\mathrm{iter}(c, s)\} & \text{(R-While)} \\
\mathrm{iter}(c, s) & \to \text{if } c \text{ then } (\mathrm{cnt}\{s\}; \mathrm{iter}(c, s)) \text{ else } () & \text{(R-Iter)} \\
\mathrm{brk}\{E_b[\mathrm{break}]\} & \to () & \text{(R-Brk)} \\
\mathrm{cnt}\{E_c[\mathrm{continue}]\} & \to () & \text{(R-Cnt)} \\
\mathrm{ret}\{E_r[\mathrm{return}\ v]\} & \to v & \text{(R-Ret)} \\
\mathrm{brk}\{v\} \to () \quad \mathrm{cnt}\{v\} \to v \quad \mathrm{ret}\{v\} \to v & & \text{(R-Frame)}
\end{array}
$$

7.3.4 Wywołania

$$
\begin{array}{lll}
f\langle \overline{\tau}, \overline{\varepsilon} \rangle \big(\overline{v}, \&\overline{\ell} \big) & \to \mathrm{ret}\Big\{ e_f[\overline{\tau}/\overline{\alpha}][\overline{v}/\overline{x}]\Big[\overline{\ell}/\overline{m}\Big] \Big\} \quad \text{gdy } \ell_i \text{ parami różne} & \text{(R-Call)} \\
(\mathrm{fn}(\overline{x} : \overline{\tau}) \Rightarrow e)(\overline{v}) & \to \mathrm{ret}\{e[\overline{v}/\overline{x}]\} & \text{(R-App)}
\end{array}
$$

Parametr `inout` jest wiązany **bezpośrednio z lokacją** wywołującego. Nie ma kopiowania przy wejściu i wyjściu: semantyka „przez lokację” jest równoważna semantyce copy-in/copy-out dzięki prawu wyłączności (lemat 8.3).

7.3.5 Sprawdzenia i pułapki

$$
\begin{array}{lll}
\langle \mathrm{check}[\kappa]\varphi, \sigma \rangle & \to \langle (), \sigma \rangle \quad \text{gdy } [\![\varphi]\!]_\sigma = \mathrm{true} & \text{(R-Chk)} \\
\langle \mathrm{check}[\kappa]\varphi, \sigma \rangle & \to \langle \mathrm{trap}(\mathrm{contract}(\kappa)), \sigma \rangle \quad \text{gdy } [\![\varphi]\!]_\sigma = \mathrm{false} & \text{(R-ChkF)} \\
\langle \mathrm{snap}[\kappa] t_0 \coloneqq t \text{ in } e, \sigma \rangle & \to \langle e[[\![t]\!]_\sigma / t_0], \sigma \rangle & \text{(R-Snap)} \\
E[\mathrm{trap}(k)] & \to \mathrm{trap}(k) \quad E \neq [] & \text{(R-Trap)}
\end{array}
$$

$[\![\varphi]\!]_\sigma$ to wartościowanie formuły, w którym lokacje są odczytywane z σ; jest zawsze zdefiniowane i obliczalne (tw. 8.8). Pułapka kończy wykonanie całego programu z kodem wyjścia zależnym od rodzaju k.

7.3.6 Kapabilności

$$
\begin{array}{lll}
\langle \mathrm{print}(\mathrm{cap}\langle \mathrm{Io} \rangle, s), \sigma \rangle & \xrightarrow{\mathrm{Io.print}(s)} \langle (), \sigma \rangle & \text{(R-Print)} \\
\langle \mathrm{read_line}(\mathrm{cap}\langle \mathrm{Io} \rangle), \sigma \rangle & \xrightarrow{\mathrm{Io.read}(r)} \langle r, \sigma \rangle \quad r \in \mathrm{Option}\langle \mathrm{Str} \rangle & \text{(R-ReadLine)}
\end{array}
$$

Wejście z otoczenia jest jedynym źródłem niedeterminizmu w obrębie aktora: dla ustalonego ciągu odpowiedzi otoczenia wykonanie jest deterministyczne.

7.3.7 Domknięcie kontekstowe

$$\frac{\langle e, \sigma\rangle \xrightarrow{\alpha} \langle e', \sigma'\rangle}{\langle E[e], \sigma\rangle \xrightarrow{\alpha} \langle E[e'], \sigma'\rangle}\ \textsc{R-Ctx}$$

7.4 Aktorzy

Konfiguracja programu to skończony multizbiór aktorów $\Pi = \{\langle a_i, e_i, \sigma_i, q_i\rangle\}$, gdzie q_i jest kolejką wiadomości (ciągiem wartości). Pamięci aktorów są rozłączne. Reguły (zapis $\Pi \parallel A$ oznacza multizbiór z wyróżnionym aktorem A):

$$\frac{\langle e, \sigma\rangle \xrightarrow{\alpha} \langle e', \sigma'\rangle}{\Pi \parallel \langle a, e, \sigma, q\rangle \xrightarrow{\alpha} \Pi \parallel \langle a, e', \sigma', q\rangle}\ \textsc{A-Local}$$

$$\frac{c = \mathrm{cap}\langle \mathrm{Spawn}\rangle \qquad b' \text{ świeży}}{\Pi \parallel \langle a, E[\mathrm{spawn}\ b(c, \overline{v})], \sigma, q\rangle \to \Pi \parallel \langle a, E[\mathrm{chan}(b')], \sigma, q\rangle \parallel \langle b', \mathrm{ret}\{e_b[\overline{v}/\overline{x}][\ell/\ \mathrm{mb}]\}, \{\ell \mapsto \mathrm{mbox}(b')\}, \varepsilon\rangle}\ \textsc{A-Spawn}$$

$$\frac{}{\Pi \parallel \langle a, E[\mathrm{send}(\mathrm{chan}(b), v)], \sigma, q\rangle \parallel \langle b, e_b, \sigma_b, q_b\rangle \to \Pi \parallel \langle a, E[()], \sigma, q\rangle \parallel \langle b, e_b, \sigma_b, q_b \cdot v\rangle}\ \textsc{A-Send}$$

$$\frac{\sigma(\ell) = \mathrm{mbox}(a)}{\Pi \parallel \langle a, E[\mathrm{recv}(\ell)], \sigma, v \cdot q\rangle \to \Pi \parallel \langle a, E[v], \sigma, q\rangle}\ \textsc{A-Recv}$$

Wysłanie do aktora zakończonego jest operacją pustą. `recv` przy pustej kolejce blokuje aktora. Ponieważ wartości nie zawierają lokacji, wysłanie wartości **kopiuje** drzewo, a aktorzy nie współdzielą stanu mutowalnego — wyścigi danych są niemożliwe z konstrukcji.

7.5 Zachowanie obserwowalne

Program startuje z konfiguracji złożonej z jednego aktora wykonującego $\mathrm{main}(\mathrm{caps})$, gdzie caps zawiera wszystkie kapabilności. **Zachowanie** programu to zbiór śladów zdarzeń obserwowalnych zakończonych jednym z: $\mathrm{exit}(n)$ (gdy main zwróci n; pozostali aktorzy są wtedy kończeni), $\mathrm{trap}(k)$, albo nieskończonych. Planista aktorów jest niedeterministyczny z założeniem słabej sprawiedliwości.

> **Uwaga.** Implementacja realizuje zapis elementu tablicy i `push` **w miejscu**. Jest to poprawne, bo w każdym osiągalnym stanie wartość przechowywana w lokacji nie jest współdzielona z żadną inną lokacją ani zmienną (wartości są kopiowane przy odczycie). Implementacja może unikać kopiowania przez zliczanie referencji z kopiowaniem przy zapisie (copy-on-write); tożsamość obserwowalna obu strategii wynika z braku lokacji w wartościach.

8 Metateoria

Rozdział formułuje gwarancje języka. Dowody mają postać szkiców; ich status i ocenę ryzyka podsumowuje §8.4. Do typowania konfiguracji czasu wykonania używa się dodatkowych reguł dla ramek: $\Gamma, \mathrm{ret} : \tau \vdash e : \tau!\varepsilon$ daje $\Gamma \vdash \mathrm{ret}\{e\} : \tau!\varepsilon$; $\Gamma, \mathrm{loop} \vdash e : \mathrm{Unit}\ !\varepsilon$ daje $\Gamma \vdash \mathrm{brk}\{e\} : \mathrm{Unit}\ !\varepsilon$ oraz $\Gamma \vdash \mathrm{cnt}\{e\} : \mathrm{Unit}\ !\varepsilon$; pamięć jest dobrze typowana ($\Lambda \vdash \sigma$), gdy $\mathrm{dom}(\sigma) = \mathrm{dom}(\Lambda)$ i $\cdot; \cdot; \Lambda \vdash \sigma(\ell) : \Lambda(\ell)!\varnothing$ dla każdej lokacji.

8.1 Bezpieczeństwo pamięci i brak aliasingu

Lemat 8.1 (brak lokacji w wartościach)**.** W każdej konfiguracji osiągalnej z programu wykonywalnego: (a) żadna wartość nie zawiera lokacji; (b) żadna wartość w σ nie zawiera lokacji; (c) ciało każdego domknięcia nie zawiera lokacji ani zmiennych mutowalnych.

Szkic dowodu. Indukcja po długości wykonania. Gramatyka wartości nie ma lokacji. Jedyną regułą wprowadzającą lokację do wyrażenia jest R-Var (podstawienie ℓ za m) i R-Call (za parametry `inout`). Podstawienie nie może umieścić lokacji w ciele domknięcia, bo na mocy T-Lam ciało domknięcia nie ma wolnych zmiennych mutowalnych. Odczyt R-Read zwraca $\sigma(\ell)$, które z (b) nie zawiera lokacji; zapisy umieszczają w σ wyłącznie wartości. □

Lemat 8.2 (injektywność wiązań mutowalnych)**.** W każdej osiągalnej konfiguracji, w obrębie każdej ramki $\mathrm{ret}\{\ldots\}$, różne zmienne mutowalne wiązane w tej aktywacji są związane z różnymi lokacjami. W szczególności warunek poboczny reguły R-Call jest zawsze spełniony dla programów dobrze typowanych.

Szkic dowodu. R-Var przydziela lokację świeżą. W R-Call parametry `inout` otrzymują lokacje $\overline{\ell}$, które są obrazami parami różnych celów $\overline{\mu}$ (przesłanka T-Call); z założenia indukcyjnego dla aktywacji wywołującej różne cele wskazują różne lokacje. □

Lemat 8.3 (własność ramki)**.** Jeśli $\langle f\big(\overline{v}, \&\overline{\ell}\big), \sigma\rangle \xrightarrow{*} \langle v, \sigma'\rangle$ w jednym aktorze, to $\sigma'(\ell) = \sigma(\ell)$ dla każdego $\ell \in \mathrm{dom}(\sigma) \setminus \big\{\overline{\ell}\big\}$. To samo dotyczy wykonań kończących się pułapką (w chwili pułapki).

Szkic dowodu. Po R-Call wyrażenie $\mathrm{ret}\big\{e_f[\ldots]\big\}$ zawiera tylko lokacje $\overline{\ell}$: ciało f jest zamknięte poza parametrami, a z lematu 8.1 podstawione wartości nie wnoszą lokacji. Każdy krok redukcji zapisuje wyłącznie lokację występującą w redeksie, a nowe lokacje są świeże. Indukcja po długości wykonania pokazuje, że zbiór lokacji w wyrażeniu jest zawsze podzbiorem $\overline{\ell}$ i lokacji świeżych. □

Lemat 8.3 jest semantycznym rdzeniem prawa wyłączności: uzasadnia kwantyfikację tylko po $\overline{\mu'}$ w regule wywołania (§6.4) oraz implementację zapisów w miejscu.

Twierdzenie 8.4 (bezpieczeństwo typów)**.** Niech $\cdot;\cdot;\Lambda \vdash e : \tau!\varepsilon$, $\Lambda \vdash \sigma$, a e nie zawiera dziur ani nieobjętych ramkami `break`, `continue`, `return`. Wtedy:

1. (*postęp*) e jest wartością, $e = \mathrm{trap}(k)$, e jest zablokowane na recv z pustą kolejką, albo istnieje krok $\langle e, \sigma\rangle \xrightarrow{\alpha} \langle e', \sigma'\rangle$;
2. (*zachowanie*) jeśli $\langle e, \sigma\rangle \xrightarrow{\alpha} \langle e', \sigma'\rangle$, to istnieje $\Lambda' \supseteq \Lambda$ takie, że $\cdot;\cdot;\Lambda' \vdash e' : \tau!\varepsilon$ i $\Lambda' \vdash \sigma'$.

Szkic dowodu. Standardowa indukcja po wyprowadzeniu typu (postęp) i po regule redukcji (zachowanie) z lematami o podstawieniu dla zmiennych niemutowalnych, mutowalnych (podstawienie lokacji) i typów. Przypadek R-Call korzysta z lematu 8.2. Przypadki sterowania korzystają z niezmiennika, że `break` występuje tylko w kontekście E_b wewnątrz ramki brk (wpis loop jest dodawany wyłącznie przez T-While i usuwany przez T-Lam), i analogicznie dla `continue` i `return`. □

8.2 Poprawność weryfikacji

Twierdzenie 8.5 (poprawność warunków weryfikacji)**.** Załóżmy, że każde zobowiązanie każdej funkcji programu ma status `proved`. Wtedy dla każdej funkcji f i każdej pamięci, w której argumenty spełniają $H \wedge P_f$, wykonanie $f\big(\overline{v}, \&\overline{\ell}\big)$:

1. nie kończy się pułapką innego rodzaju niż stack;
2. jeśli się kończy, to wartość wyniku r oraz końcowe wartości $\overline{\ell}$ spełniają Q_f (z $\mathrm{old}(\overline{m})$ równym wartościom początkowym) i hipotezy typu wyniku.

Szkic dowodu. Definiujemy semantyczną trójkę Hoare'a $\vDash \{\Phi\}e\{Q\}$ dla czwórek Q i pokazujemy indukcją po strukturze e (z indukcją po długości wykonania dla pętli i rekursji), że $\vDash \{\mathrm{wp}(e, Q)\}e\{Q\}$. Przypadek pętli: niezmiennik zachodzi na początku (`inv-init`), jest zachowywany (`inv-pres`) przy dowolnych wartościach zmiennych $\overline{w}$, a wartości spoza $\overline{w}$ nie zmieniają się w ciele (analiza syntaktyczna ws). Przypadek wywołania: postkondycja wywoływanej funkcji zachodzi z założenia indukcyjnego (po długości wykonania), a pozostała część pamięci jest niezmieniona z lematu 8.3. Na koniec: z prawdziwości wszystkich VC_κ wynika prawdziwość warunku z wszystkimi zobowiązaniami jako koniunkcjami (indukcja po liczbie zobowiązań na ścieżce: pierwsze naruszone zobowiązanie przeczyłoby odpowiadającemu mu VC_κ, którego hipotezy są wtedy spełnione).□

Twierdzenie 8.6 (poprawność weryfikacji stopniowanej)**.** Niech $P' = \mathcal{R}(\mathcal{E}(P))$ dla programu bez zobowiązań o statusie `refuted`. Wtedy każde wykonanie P' ma jedną z postaci: kończy się normalnie, kończy się pułapką $\mathrm{trap}(\mathrm{contract}(\kappa))$ lub pułapką niejawną dla zobowiązania κ o statusie `unknown` albo `refuted-inductive`, kończy się pułapką stack, albo jest nieskończone. Ponadto w każdym punkcie wykonania osiągającym zobowiązanie o statusie `proved` jego formuła jest prawdziwa.

Szkic dowodu. Przypuśćmy, że w pewnym wykonaniu formuła zobowiązania o statusie `proved` jest fałszywa. Rozważmy **pierwsze** w porządku wykonania naruszenie dowolnego zobowiązania. Nie jest to zobowiązanie sprawdzane dynamicznie, bo jego naruszenie kończy wykonanie pułapką przed kontynuacją. Jest to więc zobowiązanie κ o statusie `proved`. Hipotezy VC_κ to formuły zobowiązań napotkanych wcześniej na tej ścieżce wykonania (w tym postkondycje wywołań i niezmienniki z poprzednich obrotów pętli), które z minimalności były prawdziwe. Z tw. 8.5 (zastosowanego do wyrażeń, a nie tylko całych funkcji) w osiągniętym stanie zachodzi φ_κ — sprzeczność. Pierwsza część wynika z tw. 8.4 i z tego, że jedynymi źródłami pułapek są zobowiązania niejawne i sprawdzenia. □

To twierdzenie jest formalnym uzasadnieniem zasady Z6: niedowiedziona obietnica nigdy nie prowadzi do zachowania niezdefiniowanego, a jedynie do kontrolowanej pułapki w miejscu, które jest wskazane w raporcie kompilacji.

8.3 Efekty, terminacja, wykonywalność

Twierdzenie 8.7 (poprawność efektów i terminacja)**.** Niech $\cdot;\cdot;\Lambda \vdash e : \tau ! \varepsilon$. Wtedy:

1. każde zdarzenie $K.\mathrm{op}(\overline{v})$ emitowane w wykonaniu e spełnia $K \in \varepsilon$;
2. jeśli $\mathrm{Div} \notin \varepsilon$ i e jest wyrażeniem programu wykonywalnego, to każde wykonanie e jest skończone (kończy się wartością lub pułapką), bez względu na statusy zobowiązań `var-nn`, `var-dec` i `term`.

Szkic dowodu. (1) Przez zachowanie typów z efektami: każda reguła emitująca zdarzenie K wymaga typu $\mathrm{Cap}\langle K\rangle$ i dodaje K do efektu (T-Prim). (2) Pętle w e mają wariant (inaczej T-WhileDiv dodałoby Div). Każdy obrót albo zmniejsza nieujemną miarę całkowitą, albo — gdy odpowiednie zobowiązanie nie zostało udowodnione — wykonuje sprawdzenie, które przy naruszeniu kończy wykonanie pułapką. Analogicznie wywołania rekurencyjne w obrębie silnie spójnej składowej zmniejszają miarę lub kończą się pułapką. Wywoływane funkcje spoza składowej są totalne z założenia indukcyjnego po porządku topologicznym grafu składowych. `recv` ma efekt Div, więc nie występuje. Zatem miara leksykograficzna (pozycja w porządku topologicznym, miara wywołania, miary pętli) maleje dobrze ufundowanie; dla miar strukturalnych ranga jest liczbą naturalną malejącą przy przejściu do podstruktury (§6.4). □

Twierdzenie 8.8 (wykonywalność kontraktów)**.** Dla każdej formuły $\varphi \in F$ i każdego wartościowania jej zmiennych wolnych spełniającego $\mathrm{wd}(\varphi)$ wartość $[\![\varphi]\!]$ jest określona i obliczalna. Ponadto funkcja specyfikacyjna rekurencyjna jest dopuszczona wyłącznie wtedy, gdy jej zobowiązanie `term` ma status `proved` (w przeciwnym razie błąd `E4043`).

Szkic dowodu. Indukcja po strukturze φ. Kwantyfikatory są ograniczone do skończonych przedziałów liczb całkowitych, więc są obliczalne przez wyliczenie. Termy częściowe są określone na mocy $\mathrm{wd}(\varphi)$. Funkcje specyfikacyjne są totalne. Jeżeli zobowiązanie `wd` formuły ma status `unknown`, przekształcenie $\mathcal{R}$ wstawia sprawdzenie $\mathrm{wd}(\varphi)$ przed sprawdzeniem φ, więc wartościowanie zawsze odbywa się przy spełnionym $\mathrm{wd}(\varphi)$. □

Twierdzenie 8.9 (forma kanoniczna)**.** Dla każdego drzewa składni A: $\mathrm{parse}(\mathrm{fmt}(A)) = A$ (z dokładnością do pozycji w źródle). W konsekwencji fmt ∘ parse jest idempotentna na tekstach poprawnych składniowo.

Szkic dowodu. fmt wypisuje nawiasy wszędzie tam, gdzie priorytety z §2.4 wymagają ich do odtworzenia drzewa, i łamie wiersze tylko po tokenach, po których reguła układu (§2.2) nie wstawia średnika albo gdzie wstawiony średnik jest zgodny ze strukturą. Indukcja po A. W prototypie własność jest sprawdzana testami: dla każdego programu w repozytorium fmt zachowuje drzewo i komplet komentarzy, jest idempotentna, a losowe zmiany wcięć i odstępów między tokenami nie zmieniają wyniku (jednoznaczność postaci kanonicznej). Kompilator nie zgłasza `E0100`, jeśli fmt dla danego tekstu nie zachowałby drzewa. □

Twierdzenie 8.10 (determinizm kompilacji)**.** Lista diagnostyk, statusy zobowiązań i kod wynikowy zależą wyłącznie od tekstu programu, wersji kompilatora i jawnych opcji kompilacji.

Szkic dowodu. Wszystkie fazy (parsowanie, elaboracja, typowanie, generowanie warunków, przykłady z budżetem kroków, testy właściwości z ziarnem wyznaczonym z tekstu) są deterministyczne; Solve_β spełnia (S3); diagnostyki są sortowane według pozycji w źródle i kodu. □

8.4 Status dowodów

Wynik	Status	Ocena ryzyka
Lematy 8.1–8.2	szkic	niskie — konstrukcja standardowa dla języków z semantyką wartości i `inout` (por. Hylo, Swift)
Lemat 8.3	zmechanizowany dla fragmentu (§8.5)	niskie
Tw. 8.4	szkic	niskie — standardowe bezpieczeństwo typów z ramkami sterowania
Tw. 8.5	zmechanizowane dla fragmentu (§8.5)	niskie dla fragmentu; **średnie** dla zgodności elaboracji (tablice, domknięcia, typy funkcyjne)
Tw. 8.6	zmechanizowane dla fragmentu (§8.5)	niskie dla fragmentu; **średnie** dla granic domknięć i monitorów typów funkcyjnych
Tw. 8.7	część 2 zmechanizowana dla pętli i wywołań modularnych (§8.5)	niskie; terminacja rekursji — szkic
Tw. 8.8	szkic	niskie
Tw. 8.9–8.10	zobowiązanie implementacyjne	niskie — weryfikowalne testami właściwości implementacji
Hipoteza 6.3	otwarta	wpływa tylko na kompletność, nie na poprawność

8.5 Mechanizacja (Lean 4)

Katalog `lean/` repozytorium zawiera mechanizację (Lean 4, bez bibliotek zewnętrznych; dowody nie używają `sorry`, a jedynymi aksjomatami są `propext`, `Quot.sound` i `Classical.choice`) dla **fragmentu sekwencyjnego**, który obejmuje elementy oznaczone wyżej jako ryzyko średnie:

- instrukcje z `break`, `continue`, `return`, pętle z niezmiennikiem i wariantem, zobowiązania jawne i sprawdzenia dynamiczne oraz modularne wywołania procedur z parametrami wartościowymi i `inout`;
- każde zobowiązanie ma tryb: `proved` (naruszenie obserwowane jako wynik `fail`), `checked` (pułapka) lub `erased` (usunięte przez $\mathcal{R}$);
- semantyka z paliwem (maleje przy każdym obrocie pętli i wywołaniu), wp względem czwórki warunków końcowych; havoc pętli tylko dla zmiennych zapisywanych w ciele; wywołanie przez kontrakt z niezależnym podstawieniem celów `inout`;
- wyrażenia i formuły osadzone płytko (funkcje stanu), typ wartości dowolny.

Twierdzenie Lean	Odpowiednik
`exec_frame`	lemat 8.3 w postaci ramki zapisu: wykonanie zmienia tylko `writes s` (podstawa havoc w regule pętli i własności ramki wywołania)
`soundness`, `no_fail`	tw. 8.5: przy poprawnym VC żadne wykonanie nie narusza udowodnionego zobowiązania, a wynik spełnia odpowiednią składową czwórki warunków końcowych
`no_trap`	tw. 8.5 (1): program bez sprawdzeń dynamicznych nie kończy się pułapką
`erase_exec`, `gradual`	tw. 8.6 i przekształcenie $\mathcal{R}$: usunięcie udowodnionych zobowiązań nie zmienia wykonania; program wykonywalny kończy się zgodnie z kontraktem albo pułapką sprawdzenia
`terminates`	tw. 8.7 (2) dla pętli: przy poprawnym VC i terminujących wywoływanych procedurach każde wykonanie jest skończone, niezależnie od tego, czy wariant jest dowiedziony, czy sprawdzany

`exclusivity_necessary`	konieczność prawa wyłączności (§5.4): kontrprzykład, w którym wywołanie `f(&y, &y)` ma prawdziwy warunek weryfikacji, a wykonanie łamie warunek końcowy

Poza mechanizacją pozostają: typowanie (lematy 8.1–8.2, tw. 8.4), zgodność elaboracji pełnej składni z fragmentem (w tym tablice jako wartości, domknięcia i typy funkcyjne), dobra określoność formuł (tw. 8.8), terminacja rekursji przez miary `decreases` oraz aktorzy.

9 Diagnostyka

Format diagnostyki jest **normatywny**: jest częścią interfejsu języka, tak jak składnia. Każda diagnostyka istnieje w dwóch równoważnych postaciach: tekstowej (dla człowieka) i strukturalnej JSON (dla narzędzi i modeli). Postać tekstowa jest funkcją postaci JSON.

9.1 Rekord diagnostyki

```
Diagnostic = {
  code:        string,                // np. "E4020"; stabilny między wersjami
  severity:    "error" | "warning" | "info" | "hole",
  span:        Span,                  // miejsce główne
  spec_ref:    string[],              // sekcje tej specyfikacji, np. ["6.4", "6.6"]
  message:     string,                // jedno zdanie, bez żargonu implementacji
  expected?:   string,                // typ, formuła lub wzorzec oczekiwany
  found?:      string,                // typ, formuła lub wzorzec znaleziony
  obligation?: { id: string, kind: string, formula: string,
                 status: "proved" | "unknown" | "refuted" | "refuted-inductive" },
  counterexample?: {
                 assignment: { [name: string]: string }, // składnia Rdzenia
                 class:  "real" | "inductive",
                 replay: "confirmed" | "not-reproduced" | "not-replayable",
                 trace?: Step[] },
  goal?:       HoleGoal,              // tylko dla severity = "hole" (§9.5)
  fix?:        { patch: string,       // ujednolicony diff
                 confidence: "unique" | "suggested" },
  related:     { span: Span, message: string }[]
}
Span = { file: string, line: int, col: int, end_line: int, end_col: int }
Step = { span: Span, bindings: { [name: string]: string } }
```

Poprawka o pewności `unique` jest jedyną zmianą minimalną usuwającą diagnostykę (np. brakujący import, forma kanoniczna, brakujący przypadek wzorca z dziurą jako ciałem). Poprawka `suggested` jest jedną z wielu możliwych.

9.2 Klasy kodów

Zakres	Klasa
`E0xxx`	leksyka, składnia, forma kanoniczna (`E0100`)
`E1xxx`	nazwy, moduły, widoczność
`E2xxx`	typy kształtu, inferencja, ograniczenia `Eq`/`Ord`, operator `?` (`E2140`)
`E3xxx`	efekty i kapabilności (m.in. wywołanie funkcji częściowej z totalnej)
`E4xxx`, `W4xxx`	weryfikacja: zobowiązania obalone (`E40xx`–`E41xx`), nieznane (`W4200`), indukcyjne niejawne (`W4400`), przykłady i właściwości (`E43xx`)
`E5xxx`, `W5xxx`	wyczerpywalność (`E5001`) i nieużyteczne gałęzie (`W5002`)

`E6xxx`	prawo wyłączności, ucieczka lokacji (`E6010`), formuły spoza fragmentu (`E6101`)
`H7xxx`	raporty dziur

9.3 Brak kaskady

Gdy przesłanka reguły typowania nie zachodzi dla podwyrażenia e, kompilator zgłasza diagnostykę i zastępuje e węzłem err typowanym regułą

$$\frac{}{\Gamma \vdash \mathrm{err} : \tau ! \varnothing}\ \textsc{T-Err}$$

dla dowolnego τ (unifikującego się z każdym typem). Zmienną wiązaną wyrażeniem zawierającym err oznacza się jako **skażoną**. Diagnostyka typowania jest tłumiona, jeśli dotyczy wyrażenia zawierającego err lub zmienną skażoną. W warunkach weryfikacji err jest traktowany jak dziura ($\mathrm{wp} = \top$), więc błąd typu nie generuje wtórnych błędów weryfikacji. Kompilator zgłasza w jednym przebiegu wszystkie diagnostyki **niezależne**.

9.4 Kontrprzykłady

Kontrprzykład podaje wartości wszystkich zmiennych występujących w zanegowanym warunku weryfikacji, w składni Rdzenia (tak, by dało się je wkleić jako literały do bloku `examples`). Zmienne pochodzące z „havoc" pętli są opisane pozycją pętli; wyniki wywołań — pozycją wywołania. Dla kontrprzykładów rzeczywistych dołącza się ślad: ciąg kroków ze stanem zmiennych przy każdym rozgałęzieniu i na początku każdego obrotu pętli.

9.5 Raport dziury

Dla każdej dziury `?h` kompilator zgłasza diagnostykę o ważności `hole` zawierającą:

```
HoleGoal = {
  type:        string,               // oczekiwany typ kształtu
  goal:        string,               // predykat N(ν) z §6.4
  on_break?:   string, on_continue?: string, on_return?: string,
  hypotheses:  string[],             // fakty logiczne obowiązujące w miejscu dziury
  context:     { name: string, type: string, mutable: bool }[],
  effects:     string,               // efekty dozwolone w tym miejscu
  candidates:  { expr: string,
                 status: "satisfies" | "satisfies-if",
                 condition?: string }[]
}
```

Kandydatami są zmienne z kontekstu i wywołania funkcji o pasującym typie wyniku z argumentami będącymi zmiennymi kontekstu (głębokość 1). Dla każdego kandydata c kompilator rozstrzyga $H \wedge \text{hipotezy} \Rightarrow N(c)$ procedurą Solve_β; wynik `satisfies-if` podaje najsłabszy warunek z rodziny warunków atomowych występujących w hipotezach, przy którym implikacja zachodzi.

9.6 Raport weryfikacji

Po udanej kompilacji kompilator wypisuje raport: liczbę zobowiązań według rodzaju i statusu, listę zobowiązań `unknown` i `refuted-inductive` (czyli miejsc, w których pozostały sprawdzenia dynamiczne) oraz zaufaną bazę (§6.7) — w szczególności wszystkie funkcje `extern`, na których kontraktach opiera się wynik.

9.7 Przykłady diagnostyk

Obalenie rzeczywiste (zobowiązanie niejawne).

```
fn abs(x: Int) -> Int
    ensures result >= 0
{
```

```
    if x < 0 { -x } else { x }
}
```

```
error[E4120]: możliwe przepełnienie w negacji                [spec 6.5, 7.3]
  --> src/num.rz:4:16
   |
 4 |     if x < 0 { -x } else { x }
   |                ^^ zobowiązanie overflow#1: int_min <= -x <= int_max
   = kontrprzykład (rzeczywisty, odtworzony):
       x = -9223372036854775808
       ślad: x < 0 → true;  -x → trap(overflow)
   = poprawka (sugerowana):
       1 | fn abs(x: Int) -> Int
       2 +     requires x > int_min
```

To samo w postaci JSON (skrócone):

```
{ "code": "E4120", "severity": "error",
  "span": { "file": "src/num.rz", "line": 4, "col": 16,
            "end_line": 4, "end_col": 18 },
  "spec_ref": ["6.5", "7.3"],
  "message": "możliwe przepełnienie w negacji",
  "obligation": { "id": "abs/overflow#1", "kind": "overflow",
                  "formula": "int_min <= -x && -x <= int_max",
                  "status": "refuted" },
  "counterexample": { "assignment": { "x": "-9223372036854775808" },
                      "class": "real", "replay": "confirmed" },
  "fix": { "confidence": "suggested", "patch":
    "@@ -1 +1,2 @@\n fn abs(x: Int) -> Int\n+    requires x > int_min\n" },
  "related": [] }
```

Obalenie indukcyjne (za słaby niezmiennik).

```
fn sum_nonneg(xs: Array<Nat>) -> Nat {
    var s = 0
    for x in xs {
        s = s + x
    }
    s
}
```

```
error[E4020]: warunek końcowy może nie zachodzić               [spec 6.4, 6.6]
  --> src/sum.rz:1:34
   |
 1 | fn sum_nonneg(xs: Array<Nat>) -> Nat {
   |                                  ^^^ uściślenie wyniku: result >= 0
   = kontrprzykład (indukcyjny — stan może być nieosiągalny):
       xs = []
       s = -1      (po pętli z wiersza 3)
   = odtworzenie: xs = [] daje s = 0, naruszenie nie występuje
   = przyczyna: pętla z wiersza 3 zapisuje s, a jej niezmiennik nie mówi nic o s
   = poprawka (sugerowana):
       3 -     for x in xs {
       3 +     for x in xs
       4 +         invariant s >= 0
       5 +     {
```

W tym samym przebiegu zgłaszane jest zobowiązanie `overflow` dla `s + x`. Po dodaniu niezmiennika zostaje ono obalone rzeczywiście (np. `xs = [int_max, 1]`): suma tablicy liczb naturalnych może przekroczyć zakres `Int`, więc trzeba użyć `checked_add` (dodatek B) albo dodać warunek wstępny ograniczający sumę.

Raport dziury.

```
fn clamp(x: Int, lo: Int, hi: Int) -> Int
    requires lo <= hi
    ensures lo <= result && result <= hi
{
    if x < lo { lo } else { ?rest }
}
```

```
hole[H7001]: ?rest : Int                                        [spec 6.4, 9.5]
  --> src/clamp.rz:5:29
   = cel:        lo <= ν && ν <= hi
   = hipotezy:   lo <= hi,  !(x < lo)
   = kontekst:   x: Int, lo: Int, hi: Int
   = efekty:     ∅
   = kandydaci:  lo        spełnia
                 hi        spełnia
                 x         spełnia, jeśli x <= hi
```

Niewyczerpujące dopasowanie.

```
error[E5001]: dopasowanie nie jest wyczerpujące                 [spec 5.6]
  --> src/io.rz:12:5
   = brakujący przypadek: Err(_)
   = gałąź `Err(e) if e.code == 0` ma strażnika i nie liczy się do pokrycia
   = poprawka (unique):
      14 +         Err(_) => ?todo,
```

A Przykład kompletny: wyszukiwanie binarne

Rozdział informacyjny. Pokazuje program kanoniczny, fragment elaboracji i pełną listę zobowiązań z oczekiwanymi statusami.

A.1 Program

```
spec fn sorted(xs: Array<Int>) -> Bool =
    forall i in 0..xs.len: forall j in i..xs.len: xs[i] <= xs[j]

pub fn binary_search(xs: Array<Int>, target: Int) -> Option<Nat>
    requires sorted(xs)
    ensures match result {
        Some(i) => i < xs.len && xs[i] == target,
        None => forall k in 0..xs.len: xs[k] != target,
    }
    examples {
        binary_search([1, 3, 5], 3) == Some(1)
        binary_search([], 7) == None
        forall xs: Array<Int>, t: Int:
            !sorted(xs) || (binary_search(xs, t) != None) == contains(xs, t)
    }
{
    var lo = 0
```

```
    var hi = xs.len
    while lo < hi
        invariant 0 <= lo && lo <= hi && hi <= xs.len
        invariant forall k in 0..lo: xs[k] < target
        invariant forall k in hi..xs.len: xs[k] > target
        decreases hi - lo
    {
        let mid = lo + (hi - lo) / 2
        match compare(xs[mid], target) {
            Less => { lo = mid + 1 },
            Greater => { hi = mid },
            Equal => { return Some(mid) },
        }
    }
    None
}
```

Funkcja `contains` z testu właściwości pochodzi z preludium (dodatek B); funkcje specyfikacyjne, takie jak `sorted`, są wykonywalne (tw. 8.8), więc można je wywoływać w przykładach. Test właściwości generuje także tablice nieposortowane, dlatego ma postać implikacji.

A.2 Fragment elaboracji

Ciało pętli po elaboracji (zmienne mutowalne jako odczyty $!m$, operatory jako operacje pierwotne):

```
while lt(!lo, !hi) inv I₁, I₂, I₃ dec (hi − lo) do
  let mid = add(!lo, div(sub(!hi, !lo), 2)) in
  let o = compare(get(xs, mid), target) in
  if test(Less, o) then lo := add(mid, 1)
  else if test(Greater, o) then hi := mid
  else if test(Equal, o) then return Some(Nat)(mid)
  else trap(unreachable)
```

A.3 Zobowiązania

Nr	Rodzaj	Treść (skrót)	Status
1	`wd`	`requires`: indeksy `xs[i]`, `xs[j]` w zakresie dzięki granicom kwantyfikatorów	proved
2	`wd`	`ensures`: `xs[i]` przy `0 <= i` (hipoteza `Nat`) i `i < xs.len`	proved
3–5	`wd`	niezmienniki: indeksy w zakresach kwantyfikatorów	proved
6–8	`inv-init`	`lo = 0`, `hi = len`: zakresy kwantyfikatorów puste	proved
9	`overflow`	`hi − lo`: wynik w `[0, len]`	proved
10	`div0`	`(hi − lo) / 2`: dzielnik stały różny od zera i od `−1`	proved
11	`overflow`	`lo + (hi − lo) / 2 <= hi <= int_max`	proved
12	`bounds`	`get(xs, mid)`: `lo <= mid < hi <= len`	proved
13	`overflow`	`mid + 1 <= hi <= int_max`	proved
14–16	`inv-pres`	gałąź `Less`: `xs[k] <= xs[mid] < target` dla `k <= mid` z `sorted`	proved
17–19	`inv-pres`	gałąź `Greater`: `xs[k] >= xs[mid] > target` dla `k >= mid`	proved
20	`var-nn`	`hi − lo >= 0` z niezmiennika	proved
21–22	`var-dec`	`hi − (mid + 1) < hi − lo` oraz `mid − lo < hi − lo`	proved
23	`post`	`return Some(mid)`: `mid >= 0`, `mid < len`, `xs[mid] == target`	proved
24	`post`	wyjście z pętli: `lo = hi`, więc każdy `k` leży w jednym z zakresów	proved
25	`unreach`	dopasowanie wyczerpujące na `Ordering`	proved

26–27	`example`	oba przykłady obliczone do `true`	passed
28	`property`	256 przypadków, ziarno z tekstu funkcji	passed

Wszystkie zobowiązania należą do fragmentu F_0: kwantyfikatory w `sorted` i niezmiennikach mają postać fragmentu własności tablic (strażniki indeksowe liniowe, wartości porównywane liniowo). Program wykonywalny nie zawiera żadnego sprawdzenia dynamicznego — w szczególności dostęp `xs[mid]` jest skompilowany bez kontroli granic.

B Preludium

Rozdział normatywny. Preludium jest importowane niejawnie do każdego modułu. Funkcje wbudowane podano jako sygnatury z kontraktami, bez ciał; efekty funkcji z kapabilnościami podano jawnie. Kontrakty funkcji wbudowanych należą do zaufanej bazy (§6.7).

```
data Option<T> { None, Some(value: T) }
data Result<T, E> { Ok(value: T), Err(error: E) }
data Ordering { Less, Equal, Greater }
data Caps { Caps(io: Cap<Io>, fs: Cap<Fs>, net: Cap<Net>, clock: Cap<Clock>,
                 rand: Cap<Rand>, ffi: Cap<Ffi>, spawn: Cap<Spawn>) }

type Nat = Int where self >= 0
type Byte = Int where 0 <= self && self < 256
type Str = Array<Byte>

const int_max: Int = 0x7FFF_FFFF_FFFF_FFFF
const int_min: Int = -int_max - 1

// ---- liczby całkowite ----
fn compare(a: Int, b: Int) -> Ordering
    ensures match result { Less => a < b, Equal => a == b, Greater => a > b }
fn min(a: Int, b: Int) -> Int
    ensures result <= a && result <= b && (result == a || result == b)
fn max(a: Int, b: Int) -> Int
    ensures result >= a && result >= b && (result == a || result == b)
fn checked_add(a: Int, b: Int) -> Option<Int>
    ensures match result { Some(c) => c == a + b,
                           None => a + b > int_max || a + b < int_min }
fn checked_mul(a: Int, b: Int) -> Option<Int>        // analogicznie
fn wrapping_add(a: Int, b: Int) -> Int               // arytmetyka modulo 2^64

// ---- tablice ----
fn push<T>(inout a: Array<T>, v: T)
    requires a.len < int_max
    ensures a.len == old(a).len + 1 && a[a.len - 1] == v
    ensures forall k in 0..old(a).len: a[k] == old(a)[k]
fn pop<T>(inout a: Array<T>) -> Option<T>
    ensures match result {
        None => old(a).len == 0 && a.len == 0,
        Some(v) => a.len == old(a).len - 1 && v == old(a)[a.len]
                   && forall k in 0..a.len: a[k] == old(a)[k],
    }
fn filled<T>(n: Nat, v: T) -> Array<T>
    ensures result.len == n && forall k in 0..n: result[k] == v
fn contains<T: Eq>(a: Array<T>, v: T) -> Bool
    ensures result == exists k in 0..a.len: a[k] == v
fn slice<T>(a: Array<T>, from: Nat, to: Nat) -> Array<T>
```

```
    requires from <= to && to <= a.len
    ensures result.len == to - from
    ensures forall k in 0..result.len: result[k] == a[from + k]

// ---- tablice i napisy (Str = Array<Byte>; porównanie napisów: ==) ----
fn concat<T>(a: Array<T>, b: Array<T>) -> Array<T>       // czysta, dozwolona w
formułach
    ensures result.len == a.len + b.len
    ensures forall k in 0..a.len: result[k] == a[k]
    ensures forall k in 0..b.len: result[a.len + k] == b[k]
// slice (wyżej) jest czysta i dozwolona w formułach; jej warunek wstępny jest
// w formułach zobowiązaniem `wd`

// ---- klasy znaków ASCII (czyste, dozwolone w formułach) ----
fn is_digit(c: Int) -> Bool       // 48 <= c <= 57
fn is_lower(c: Int) -> Bool       // 97 <= c <= 122
fn is_upper(c: Int) -> Bool       // 65 <= c <= 90
fn is_alpha(c: Int) -> Bool       // is_lower(c) || is_upper(c)
fn is_space(c: Int) -> Bool       // c ∈ {32, 9, 10, 13}

// ---- liczby zmiennoprzecinkowe (poza logiką) ----
fn f64_add(a: F64, b: F64) -> F64                    // oraz sub, mul, div, sqrt
fn f64_eq(a: F64, b: F64) -> Bool                    // IEEE 754: NaN != NaN
fn int_to_f64(n: Int) -> F64
fn f64_to_int(x: F64) -> Option<Int>                 // None dla NaN i poza zakresem

// ---- kapabilności ----
fn print(io: Cap<Io>, s: Str) ! {Io}
fn read_line(io: Cap<Io>) -> Option<Str> ! {Io}
fn read_file(fs: Cap<Fs>, path: Str) -> Result<Str, IoError> ! {Fs}
fn write_file(fs: Cap<Fs>, path: Str, data: Str) -> Result<Unit, IoError> ! {Fs}
fn now(c: Cap<Clock>) -> Int ! {Clock}               // milisekundy od epoki
fn random(r: Cap<Rand>, lo: Int, hi: Int) -> Int ! {Rand}
    requires lo < hi
    ensures lo <= result && result < hi

data IoError { NotFound, PermissionDenied, Other(code: Int) }
```

Typy `Cap<K>`, `Chan<M>`, `Mailbox<M>` nie mają konstruktorów dostępnych w programie. Implementacja może dodawać do preludium funkcje, jeśli każda z nich ma kontrakt w powyższej postaci i jest zweryfikowana albo oznaczona jako zaufana w raporcie (§6.7).